

전력 피크 예측 기반 태양광 전환 시스템

교육 과정

융합_[ESG] AIoT를 활용한

스마트 환경공정제 (MES, ERP, PLC)

TEAM : 완자 IoT

팀장 : 이제우 팀원 : 김강수

프로젝트 기간

2025.05.12 ~ 2025.06.17



선순환 구조를 통한 지속가능한 에너지 솔루션

목차

1 개요

- 아이템 소개
- 선순환 구조 소개
- 팀원소개
- 비즈니스 모델
- 서비스 차별점
- ESG
- 요구사항 정의서

2 서론

- 기술 스택 요약
- Flow chart
- WBS

3 설계

- 데이터 전처리 과정
- 데이터 베이스 설계 과정
- 모델 설계 과정
- 웹 설계 과정

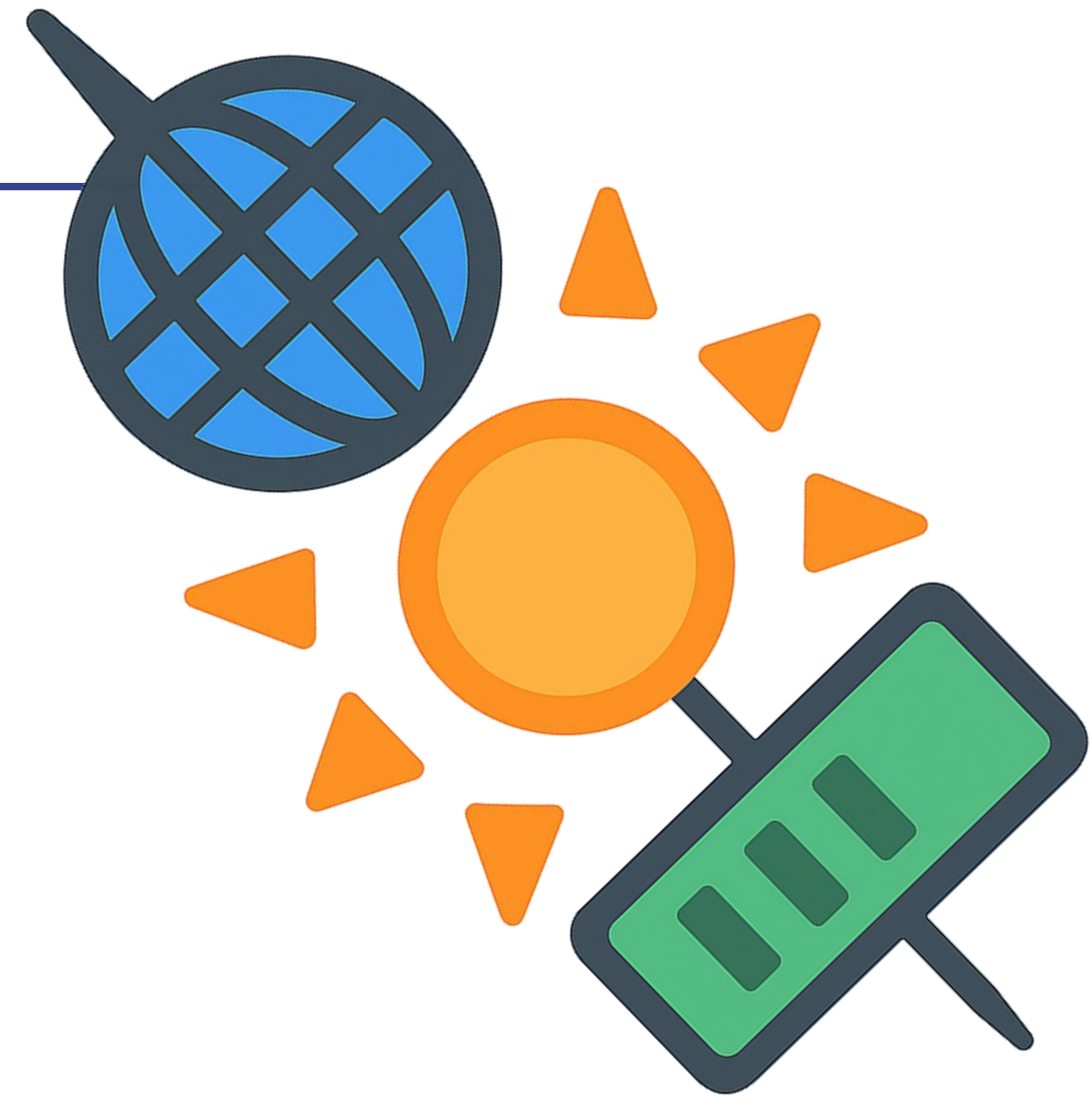
4 본론

- 시스템 구성도
- 서버
- 클라이언트
- 데이터 베이스
- 머신러닝 모델 결과
- 시연 영상

5 결론

- 시행 착오
- 향후 발전 계획
- 미래의 완자 IoT 소개
- 마무리
- 느낀점
- 출처
- Q&A

개요



1

팀장 이제우

담당 부분 :

1. 기획 총괄
2. 데이터 베이스 저장 및 관리
3. Flask 기반 웹 개발 및 시각화
4. 데이터 수집 및 제공
5. 사용자 요구사항 분석 및 기능 정의
6. 향후 발전 방향 제시
7. 프로젝트 발표 자료 구성 및 발표 담당

2

팀원 김강수

담당 부분 :

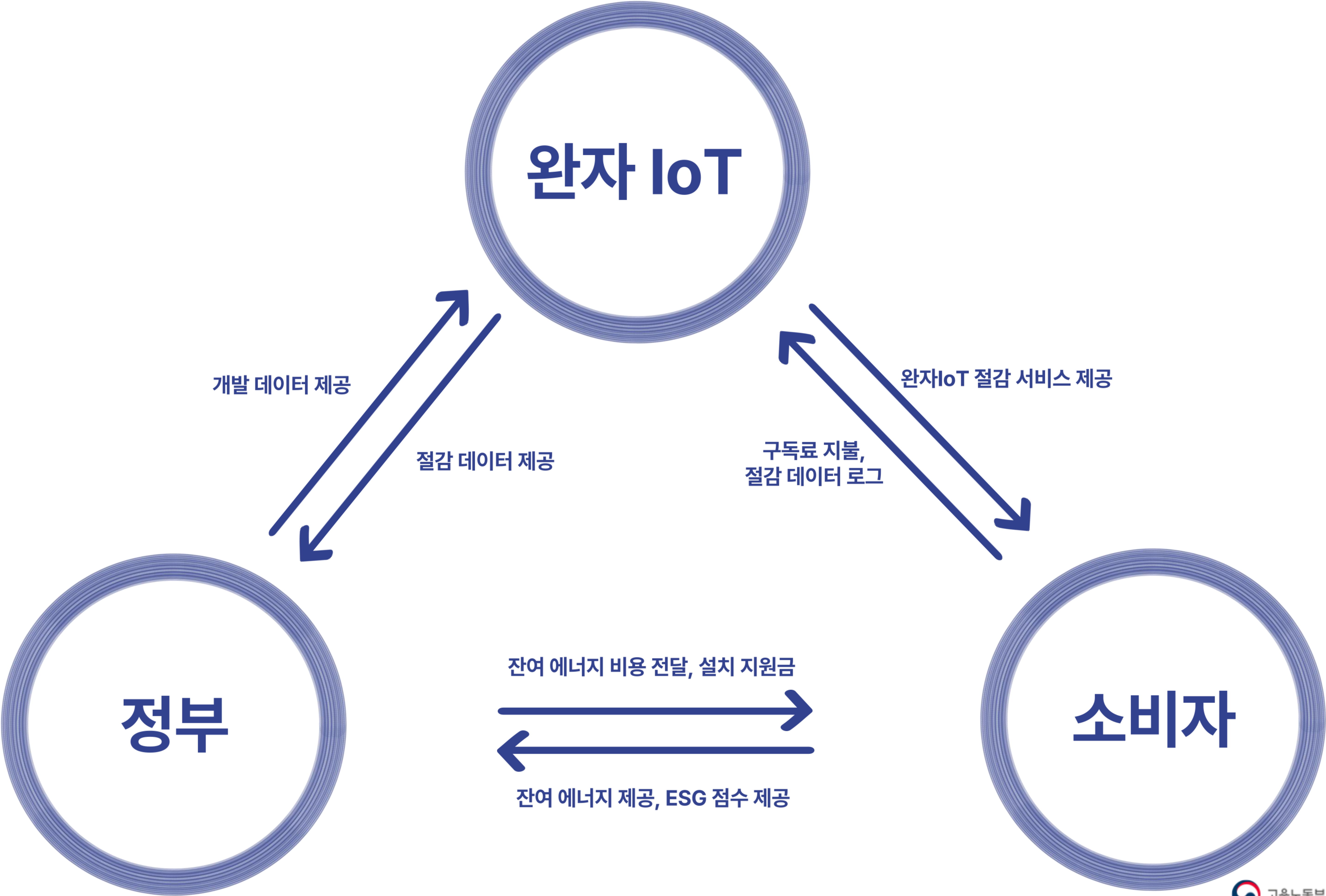
1. AI 예측 모델 구조 설계 및 최적화
2. 데이터 전처리 및 정규화 파이프라인 개발
3. 모델 학습 및 검증 과정 수행
4. 결과 분석 및 성능 지표 정리
5. PPT 및 보고서 작성
6. 모델 배포를 위한 파일 패키징 및 테스트
7. 모델 개선안 기획

전력피크시간을 예측하여 전력 피크시간때 전력공급원을 태양광 비축 에너지로 전환하여
에너지 및 비용 절감과 정전대비 전력공급 안전성을 향상시켜주는 AIOT시스템

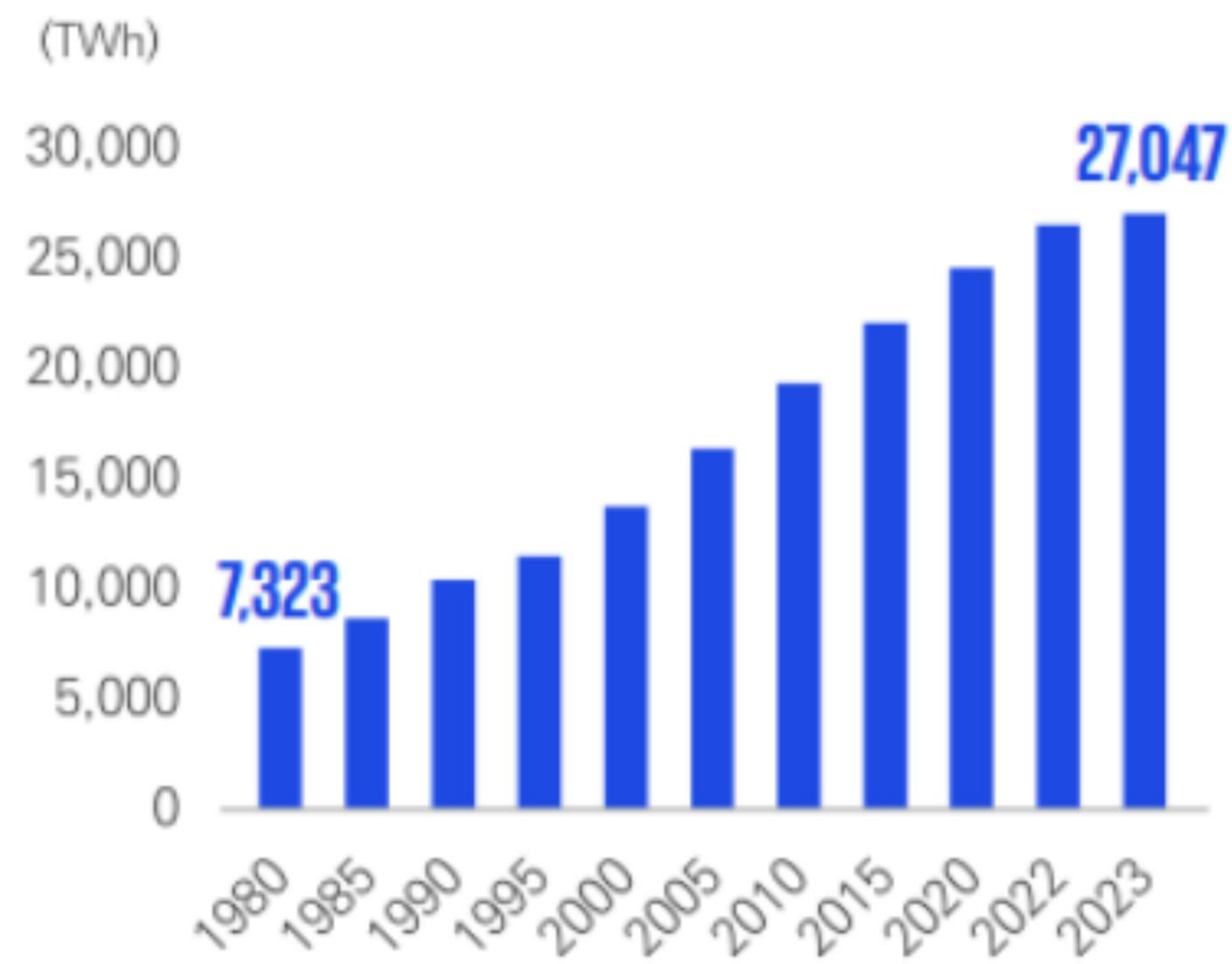


선순환 구조 소개

개요



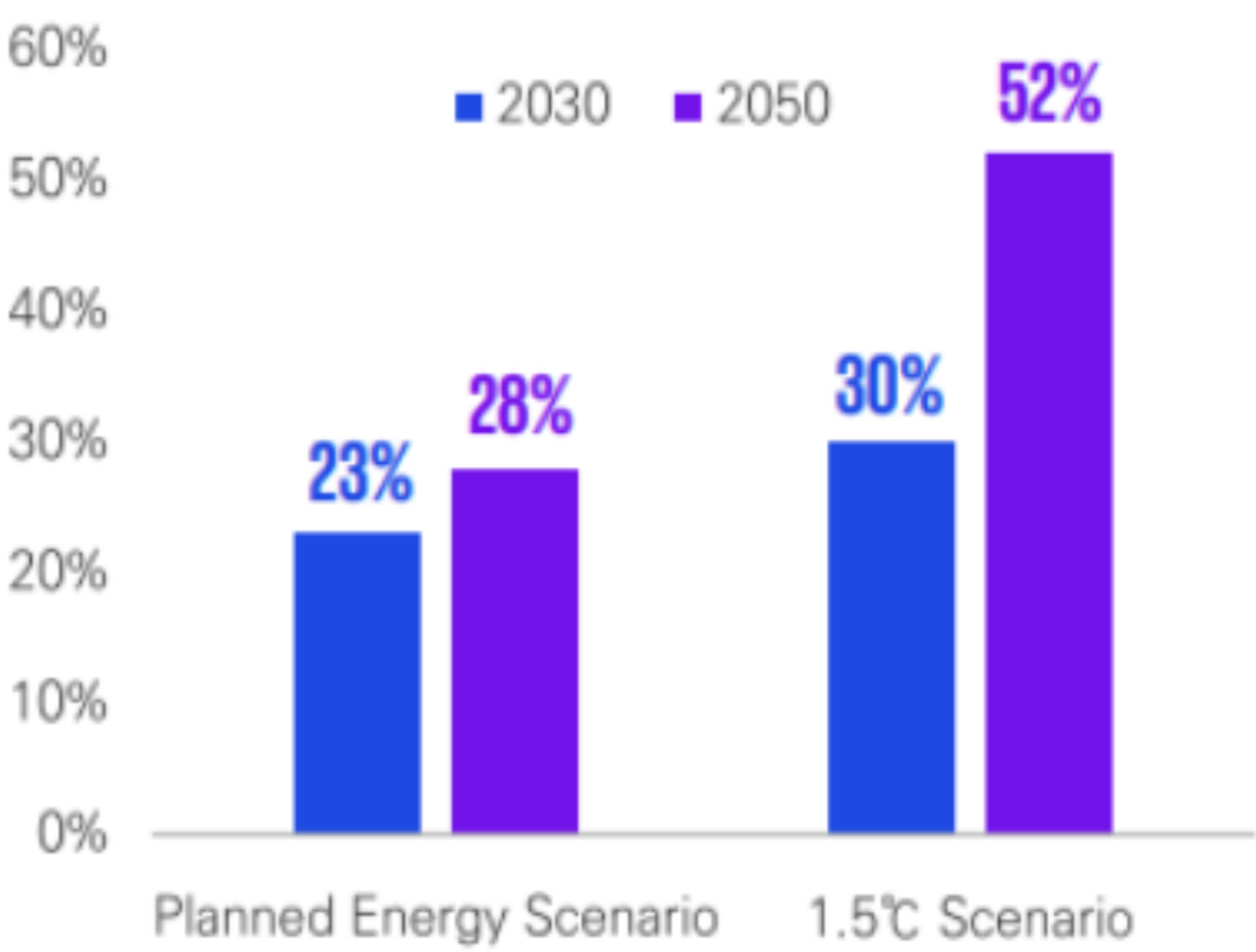
1980~2023년 전 세계 순전력소비량



자료1 출처: IEA(2025.6.20 검색 기준)

전 세계 순전력 소비 매년 사상 최고치 경신
세계 순전력 소비 40년간 약 4배 증가
AI·전기차 확산으로 전력 수요 증가세 더욱 가속화

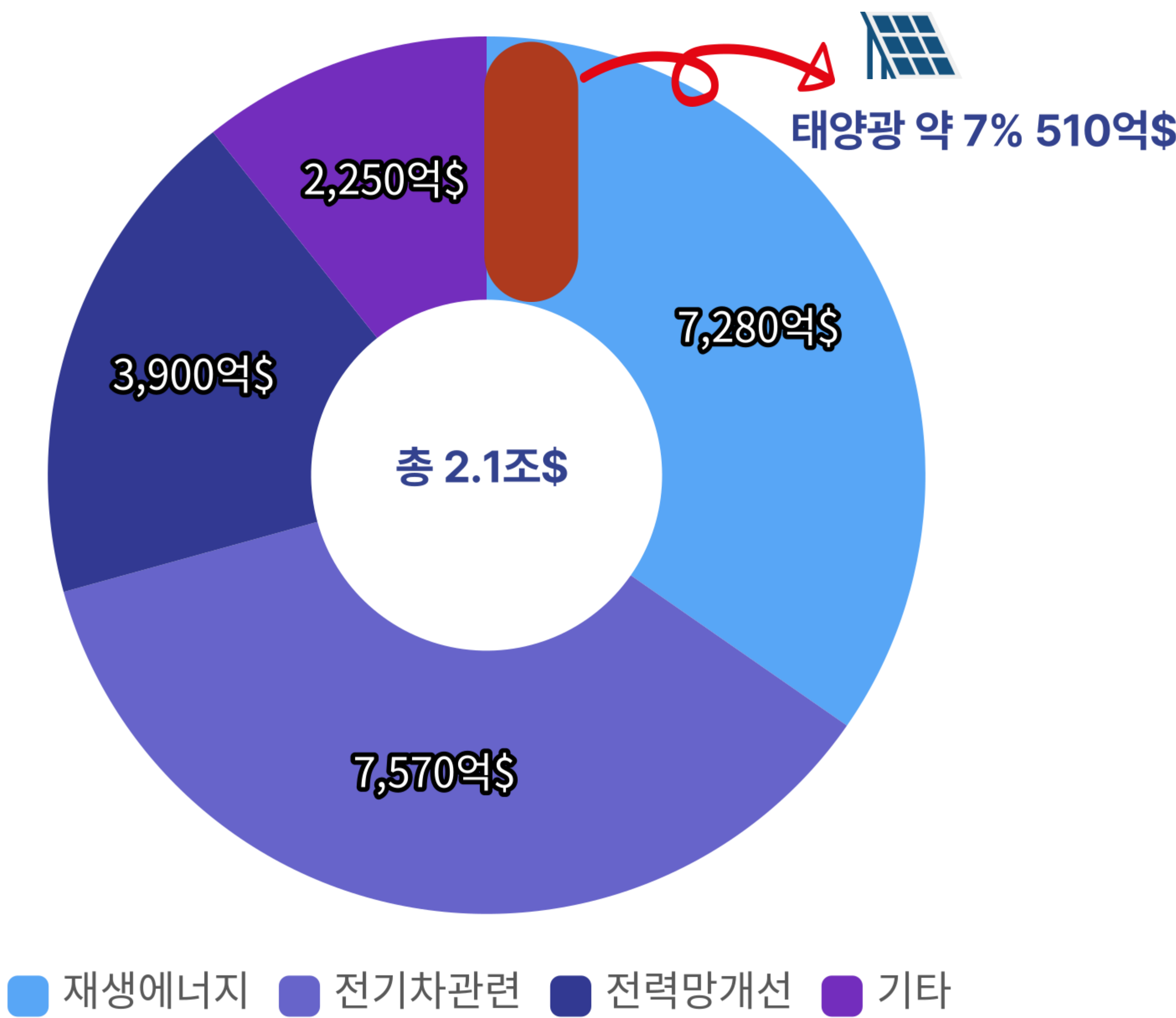
2030년·2050년 최종 에너지 수요 중 전기 비중



자료2 출처: IRENA World Energy Transition Outlook 2024

전세계 전기 비중 2030년 23~30% → 2050년 최대 52%
기후변화 1.5°C 목표 달성 위해 전기화 속도 높여야
IRENA "탄소중립엔 전기 중심 구조 전환 필수"

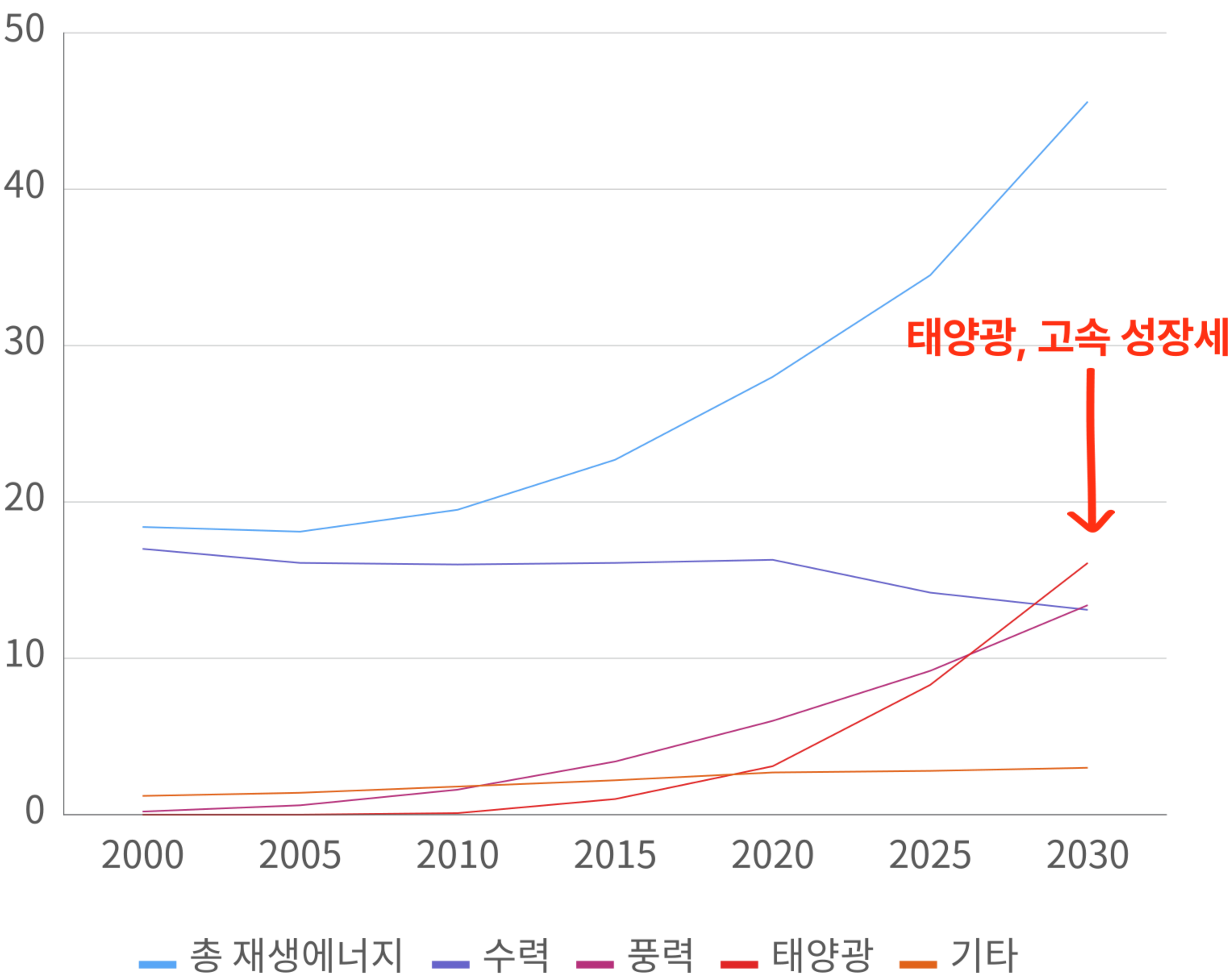
전세계 2024 에너지 전환 투자금액



자료3 출처: BloombergNEF ETI 2025

2024년 저탄소 에너지 전환 투자 사상 첫 **2조 달러 돌파**
전력망, 전기차, 재생에너지 전반에 걸쳐 급증한 자본 유입
탄소중립을 위한 **전력 수요 전환 가속화**

태양광, 미래 재생에너지 주도 기술로 부상



자료 4출처: IEA Renewables 2024

2030까지 전력 공급원 중 재생에너지 비율 약 45%추정
재생에너지 설비 증가분의 **80%가 태양광**일 것
태양광 시장이 미래 재생에너지 시장을 견인할것으로 예상

세계의 흐름에 따라가는 현 정부 2035년까지 재생에너지 비율 40% 목표

한국 전체시장 규모

대한민국 인구 약 5100만명
한전 매출 약 **94조 원**

선택시장규모

태양광 발전소: 약 17만개소
태양광 발전 도입 가구 약 **48만 호**
금액: 약 **3조 4000억 원**

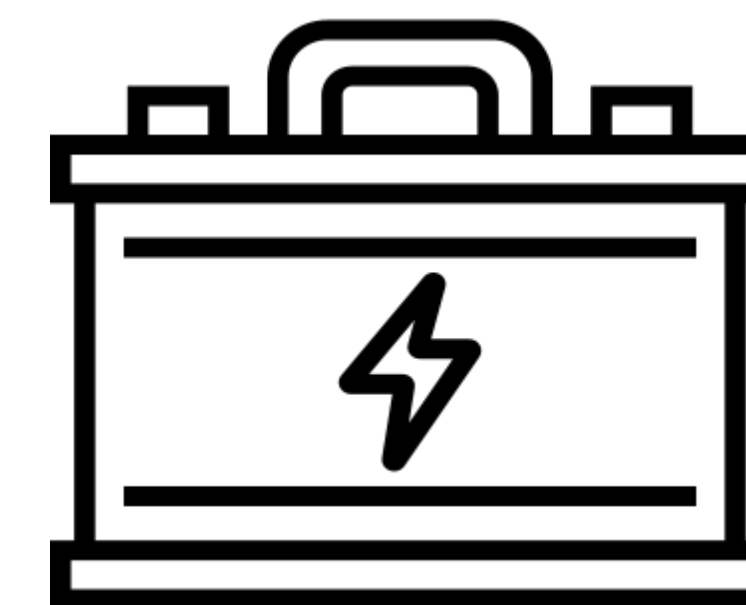
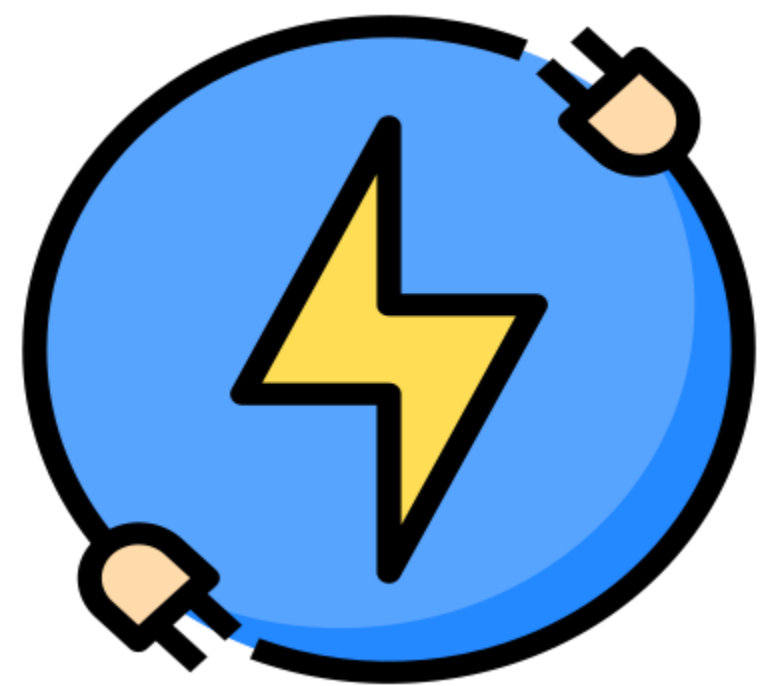
타겟소비자규모

태양광+ESS(배터리)
사용자+사용 예정자 약 **30만 호**

예상순이익

LG전자, 해준 등 경쟁사
초기 시장 점유5~12% 근거
사용자+사용 예정자 중 10%
3만 호 확보 목표
호 당 구독료 **9,900원**

= 연간 약 3억 원



수요가 현실이 되기 전, 준비된 전략이 차이를 만듭니다.

서비스 차별점

개요

기존 서비스 : 고정 동작 조건	
기존 시스템 한계	문제점
정해진 시간에만 작동	고정 스케줄 제어로 실시간 반응 불가, 유연성 없음
날씨·수요 변화 대응 불가	기온·습도·수요 급변 시 에너지 낭비 초래
실시간 피크 예측 불가능	SMP 급등 등 사전 예측 불가 → 요금 폭등 대응 실패
사용자 수동 개입 필요	자동화 부족, 사용자가 매번 전환 조작해야 함
설정값 중심의 낮은 효율성	입력값 기반 고정 루틴 → 환경 변화 대응 어려움
스마트 기술 연동 부족	IoT/AI 통합 어려움 → 확장성 낮음
태양광 에너지 중심 불안정한 전력공급	태양광 비축 에너지 고갈시 정전 위험



완자 IoT : 동적 예측 시스템	
보완점	
실시간 기반 자동 제어	시계열 예측 기반 유연한 작동 전환 가능
기상 예측, 수요 분석 통합	날씨/수요 변화 실시간 반영 최적 제어
피크 패턴 학습 기반 자동 예측	피크 발생 전 예측 후 대응 가능
전력피크 예측 시 자동 에너지 전환	사용자 개입 최소화, 무인 자동 작동
예측기반 유동 제어 알고리즘 적용	실시간 최적화 운영으로 효율 향상
IoT 기반 예측 플랫폼 완비	AI 모델 + IoT 센서 기반 동적 확장 가능
피크시간에만 태양광 ESS 전환 안정적 전력공급	피크시간에만 태양광 비축분 사용, 전력안정성 확보

한국 ESG기준원 평가 방법

대분류	중분류	소분류	대분류	중분류	소분류	
E (환경)	리더십, 거버넌스	환경경영 거버넌스	S (사회)	리더십, 거버넌스	사회책임 거버넌스 체계	
		책임 있는 경영체계 구축		전략 및 방침	사회책임 계획 및 실행방침	
	전략, 목표	환경 전략 계획		노동관행	공정고용 및 차별금지	
		온실가스 감축 목표 설정			근로자 역량개발	
	환경경영 내재화	조직환경책임 구현			일·생활 균형	
		실행체계 운영		노사관계 관리		
	위험관리	환경 리스크 식별		위험관리	안전·보건 거버넌스	
		대응 체계 구축			위험관리 체계	
	운영성과	기후변화 관리		인권	인권정책 및 거버넌스	
		자원·에너지 효율성			인권위험관리	
		물·토양·생물다양성 보전		공정운영 관행	공정운영 거버넌스	
		오염물질·화학물질 관리			공급망 리스크 관리	
		친환경 공급망		동반성장 실행	소비자 책임경영	소비자 권익 보호 거버넌스
		친환경 제품·서비스		소비자 피해 예방·보상		
		이해관계자 소통		소비자 소통 채널		
		환경정보 공개		환경성과 지표 공개 및 투명성 확보	정보보호 및 개인정보 보호	거버넌스 체계
						개인정보 위험관리
		보안 투자				
이해관계자 소통		이해관계자 소통	지역사회 거버넌스			
			위험식별 및 관리			
			성과보고			
		이해관계자 소통	사회정보 공개 체계 구축			

대분류	중분류	프로젝트 적용 포인트
E (환경)	전략, 목표	온실가스 감축 목표 : 전력 피크 시간대 예측 → 불필요한 발전 회피 → 간접적 온실가스 감축 효과
	위험관리	대응 체계 구축 : 실시간 예측을 통한 전력 피크 선대응 → 이상 상황 대응 기반
	운영성과	자원·에너지 효율성 : 수요 기반 예측 → 에너지 효율적 운영 유도
		친환경 제품·서비스 : IoT+AI 기반 예측 서비스 → 수요반응형 친환경 시스템
	이해관계자 소통	환경 분야 이해관계자 의견 반영 : 사용자/관리자 이원화 시스템, 실시간 알림 공유 등 피드백 수용 가능 구조
S (사회)	전략 및 방침	사회책임 계획 및 실행방침 수립 : SMP 예측을 통한 전력 효율화 및 지역 전력 수급 안정화 목적 명확
	위험관리	위험관리 체계 : 요금 급등·정전 위험 등 과부하 리스크 예측 → 간접적 안전 보장
	정보보호 및 개인정보 보호	보안 체계 수립 : MariaDB 기반 사용자 예측 이력 및 로그 보안 → 민감 데이터 안전 저장
	지역사회 참여 및 개발	지역사회 거버넌스 : 지역별 기상·수요 기반 SMP 예측 체계 → 전력 위기 대응용 공공 데이터 플랫폼 가능
		위험설계 및 관리 : 정전, 과부하 예측 및 대응을 위한 시계열 로깅 및 데이터 기반 의사결정 가능
	이해관계자 소통	사회정보 공개 체계 구축 : SMP 예측 결과 및 정확도 공개 → 사용자에게 투명한 피드백 제공 가능

국제 MSCI ESG평가 방법

대분류	중분류	소분류	대분류	중분류	소분류
E (환경)	기후변화 대응	탄소 배출 관리	S (사회)	인적 자원 관리	직원 건강·안전 관리
		온실가스 취약성 관리			역량 개발 및 교육
		환경 영향 재무 노출		노동 관계	노동 환경 및 갈등 관리
		제품별 탄소발자국			협력사 노동권 보장
	자원 및 천연자본	생물다양성 및 토지 이용		제품 책임	제품 안전성 및 품질
		원자재 조달 관리			화학물질 안전성
		수자원 스트레스 대응		소비자 보호	금융 소비자 보호
	오염 및 폐기물 관리	유해 물질 배출 및 오염		개인정보 보호	데이터 보안 및 프라이버시 관리
		전자폐기물 처리			신뢰도 높은 예측 결과 제공
		포장재 및 플라스틱 폐기		책임 투자	지속가능 금융 정보 활용
		유독물질 및 폐기물			지역 데이터 기반 대응 체계
	환경 기회 분야	친환경 기술 기회		논란 있는 조달	환경·인권 기준 위반 조달 방지
		친환경 건축 기회			에너지 접근성 향상 기여
		재생에너지 활용 기회		사회 기회	공공 정보의 개방성 확보
	기후 리스크 관리	전력 수요 급등에 대한 예측			사회 기반시설 최적화 예측
		에너지 효율성		지역 리스크 대응	정전/과부하 리스크 분석 및 대응
	에너지 효율성	에너지 절약 기반 예측 체계			



대분류	중분류	프로젝트 적용 포인트
E (환경)	기후 리스크 관리	전력 수요 급등에 대한 예측 대응 : SMP 예측으로 피크 시간 선제 대응
	에너지 효율성	대응 체계 구축 : 실시간 예측을 통한 전력 피크 선대응 → 이상 상황 대응 기반
	환경 기회 분야	자원·에너지 효율성 : 수요 기반 예측 → 에너지 효율적 운영 유도
		친환경 제품·서비스 : IoT+AI 기반 예측 서비스 → 수요반응형 친환경 시스템
	오염 및 폐기물 관리	환경 분야 이해관계자 의견 반영 : 사용자/관리자 이원화 시스템, 실시간 알림 공유 등 피드백 수용 가능 구조
S (사회)	개인정보 보호	데이터 보안 및 프라이버시 관리 : MariaDB 보안 설계
	소비자 경험	신뢰도 높은 예측 결과 제공 : 예측 정확도 기반 투명한 전력 정보 제공
	책임 투자	지속가능 금융 정보 활용 : SMP 예측 데이터 활용한 전력 정책 수립 근거
	지역사회 관계	지역 데이터 기반 대응 체계 : 기상+수요 기반 지역 맞춤 전력 예측
	사회 기회	사회 기반시설 최적화 예측 : 예측 기반 배전 설비 최적 운영 지원
	지역 리스크 대응	정전/과부하 리스크 분석 및 대응 : 실시간 리스크 탐지 및 시각화 제공

요구사항 정의서

개요

한국 ESG기준원 평가 방법

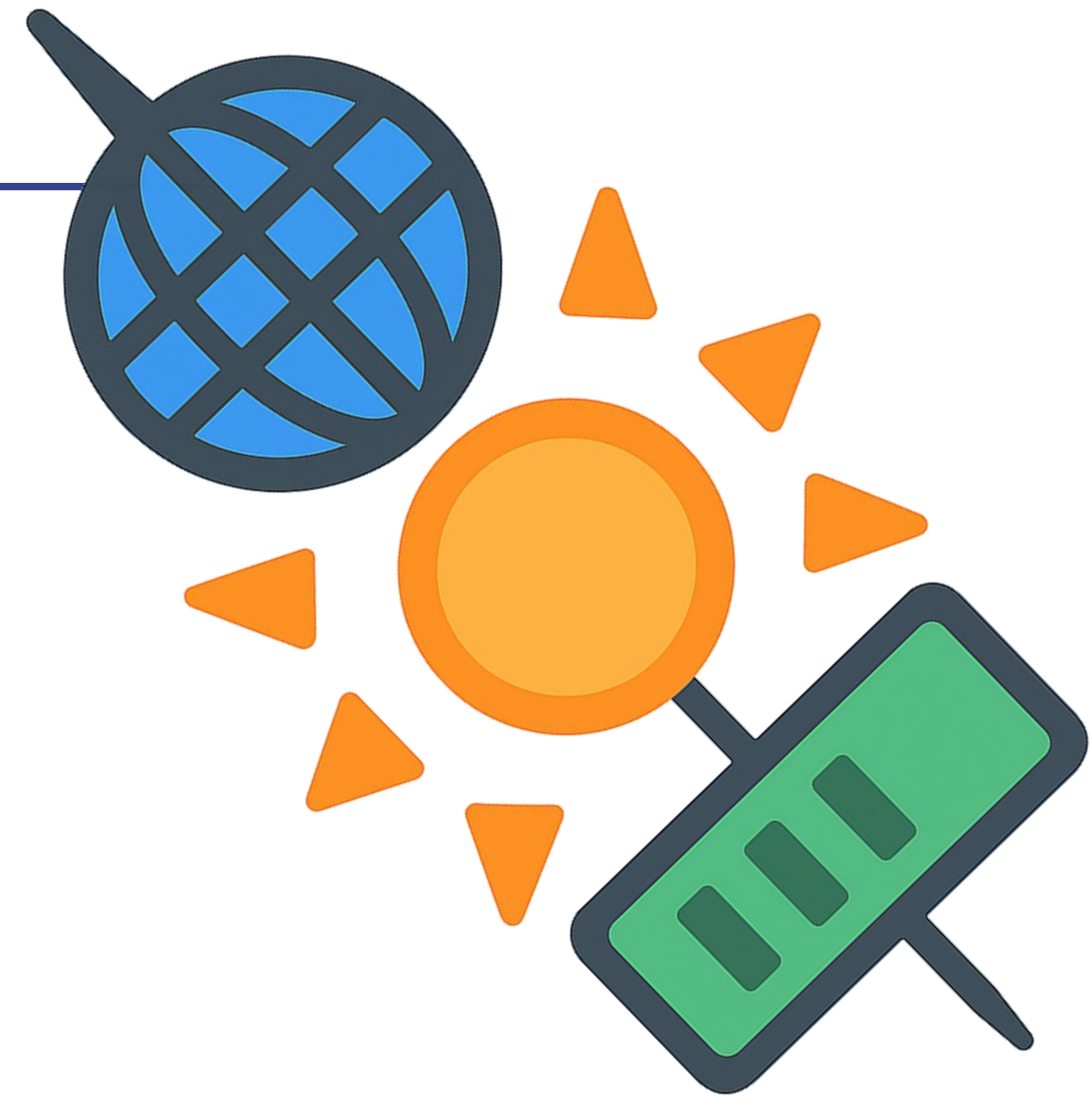
대분류	중분류	프로젝트 적용 포인트
E (환경)	전략, 목표	온실가스 감축 목표 : 전력 피크 시간대 예측 → 불필요한 발전 회피 → 간접적 온실가스 감축 효과
	위험관리	대응 체계 구축 : 실시간 예측을 통한 전력 피크 선대응 → 이상 상황 대응 기반
	운영성과	자원·에너지 효율성 : 수요 기반 예측 → 에너지 효율적 운영 유도 친환경 제품·서비스 : IoT+AI 기반 예측 서비스 → 수요반응형 친환경 시스템
	이해관계자 소통	환경 분야 이해관계자 의견 반영 : 사용자/관리자 이원화 시스템, 실시간 알림 공유 등 피드백 수용 가능 구조
S (사회)	전략 및 방침	사회책임 계획 및 실행방침 수립 : SMP 예측을 통한 전력 효율화 및 지역 전력 수급 안정화 목적 명확
	위험관리	위험관리 체계 : 요금 급등·정전 위험 등 과부하 리스크 예측 → 간접적 안전 보장
	정보보호 및 개인정보 보호	보안 체계 수립 : MariaDB 기반 사용자 예측 이력 및 로그 보안 → 민감 데이터 안전 저장
	지역사회 참여 및 개발	지역사회 거버넌스 : 지역별 기상·수요 기반 SMP 예측 체계 → 전력 위기 대응용 공공 데이터 플랫폼 가능

국제 MSCI ESG평가 방법

대분류	중분류	프로젝트 적용 포인트
E (환경)	기후 리스크 관리	전력 수요 급등에 대한 예측 대응 : SMP 예측으로 피크 시간 선제 대응
	에너지 효율성	대응 체계 구축 : 실시간 예측을 통한 전력 피크 선대응 → 이상 상황 대응 기반
	환경 기회 분야	자원·에너지 효율성 : 수요 기반 예측 → 에너지 효율적 운영 유도 친환경 제품·서비스 : IoT+AI 기반 예측 서비스 → 수요반응형 친환경 시스템
	오염 및 폐기물 관리	환경 분야 이해관계자 의견 반영 : 사용자/관리자 이원화 시스템, 실시간 알림 공유 등 피드백 수용 가능 구조
S (사회)	개인정보 보호	데이터 보안 및 프라이버시 관리 : MariaDB 보안 설계
	소비자 경험	신뢰도 높은 예측 결과 제공 : 예측 정확도 기반 투명한 전력 정보 제공
	책임 투자	지속가능 금융 정보 활용 : SMP 예측 데이터 활용한 전력 정책 수립 근거
	지역사회 관계	지역 데이터 기반 대응 체계 : 기상+수요 기반 지역 맞춤 전력 예측
	사회 기회	사회 기반시설 최적화 예측 : 예측 기반 배전 설비 최적 운영 지원
	지역 리스크 대응	정전/과부하 리스크 분석 및 대응 : 실시간 리스크 탐지 및 시각화 제공

대분류	중분류	적용 기술 및 구현 방식 요약
E (환경)	전략 및 목표 (한국)	전력 수요 예측 → 피크 회피 → 발전량 감축 → 간접 온실가스 저감
	기후 리스크 관리 (MSCI)	TCN 기반 시계열 모델로 급등 시점 사전 예측 → ESS 연계 가능성 확보
	위험관리 (한국)	실시간 예측 + 과거 수요 변화량 학습 → 이상 피크 조건에서 알림 및 자동 대응 시뮬레이션
	에너지 효율성 (MSCI)	불필요한 발전 억제 로직 → 예측 기반으로 태양광/ESS 출력 우선 적용 가능
	운영성과 (한국)	전력 수요/기온/습도 기반 피크시간대 운영 최적화 → 예측 기반 자원 분산 운용 시뮬레이션
	환경 기회 분야 (MSCI)	IoT 기반 실시간 기온·습도 수집 → MLP 예측과 연동된 전력 최적 분배 가능
	오염 및 폐기물 관리 (MSCI)	SMP 예측 정확도 향상을 통한 불필요한 발전 감소 → 간접적 탄소배출 억제 유도
S (환경)	이해관계자 소통 (한국+MSCI)	실시간 예측 결과 + 신뢰도 시각화 → 사용자/운영자 피드백 시스템 설계 및 응답속도 개선
	전략 및 실행방향 (한국)	지역 기반 전력 수요/습도 데이터 분석 → 맞춤형 피크 대응 정책 기반 제공
	위험관리 (한국)	특정 시간대 수요·기온 이상 급등 조건 탐지 → 위험 알림/자동화 대응 시나리오 설계
	정보보호 및 개인정보 보호 (한국+MSCI)	MariaDB 기반 사용자별 예측 로그 저장 + 접속 이력 암호화 → 보안성 강화
	소비자 경험 (MSCI)	확률 기반 예측 결과 + 시각적 신뢰도 표시 → 전력 사용자에게 직관적 정보 전달 UX
	책임 투자 (MSCI)	ESG 투자 대응 근거 확보 → 예측 모델 성능지표(R² 등) 및 자동화 구조 문서화 가능
	지역사회 참여 및 개발 (한국)	기온·습도·수요 기반 지역 맞춤 SMP 예측 → 지자체 대상 API/시스템화 가능
	지역사회 관계 (MSCI)	시간대별 수요예측 기반 → 지역민 전력 소비 안정성 확보 자료 제공 가능
	지역 리스크 대응 (MSCI)	급격한 기온/습도 변화 기반 예측 → 지역 전력 불안정성 사전 예측 및 알림 시스템 구축

서론



교육과정 기술스택 일람표

Python, Flask, HTML, CSS, JavaScript, Google Colab
Java, Pandas, NumPy, TensorFlow, Keras
Scikit-learn, Raspberry Pi, OlacleDB, MariaDB, Matplotlib,
Firebase, Armazon aws, XG5000, XP-builder,
Mosquitto MQTT, Arduino uno

프로그래밍

Python
JavaScript
HeidSQL

머신러닝/딥러닝

TensorFlow
Keras
scikit-learn
Huber Loss

데이터 처리 및 분석

NumPy
Pandas
RobustScaler

시각화 & 웹

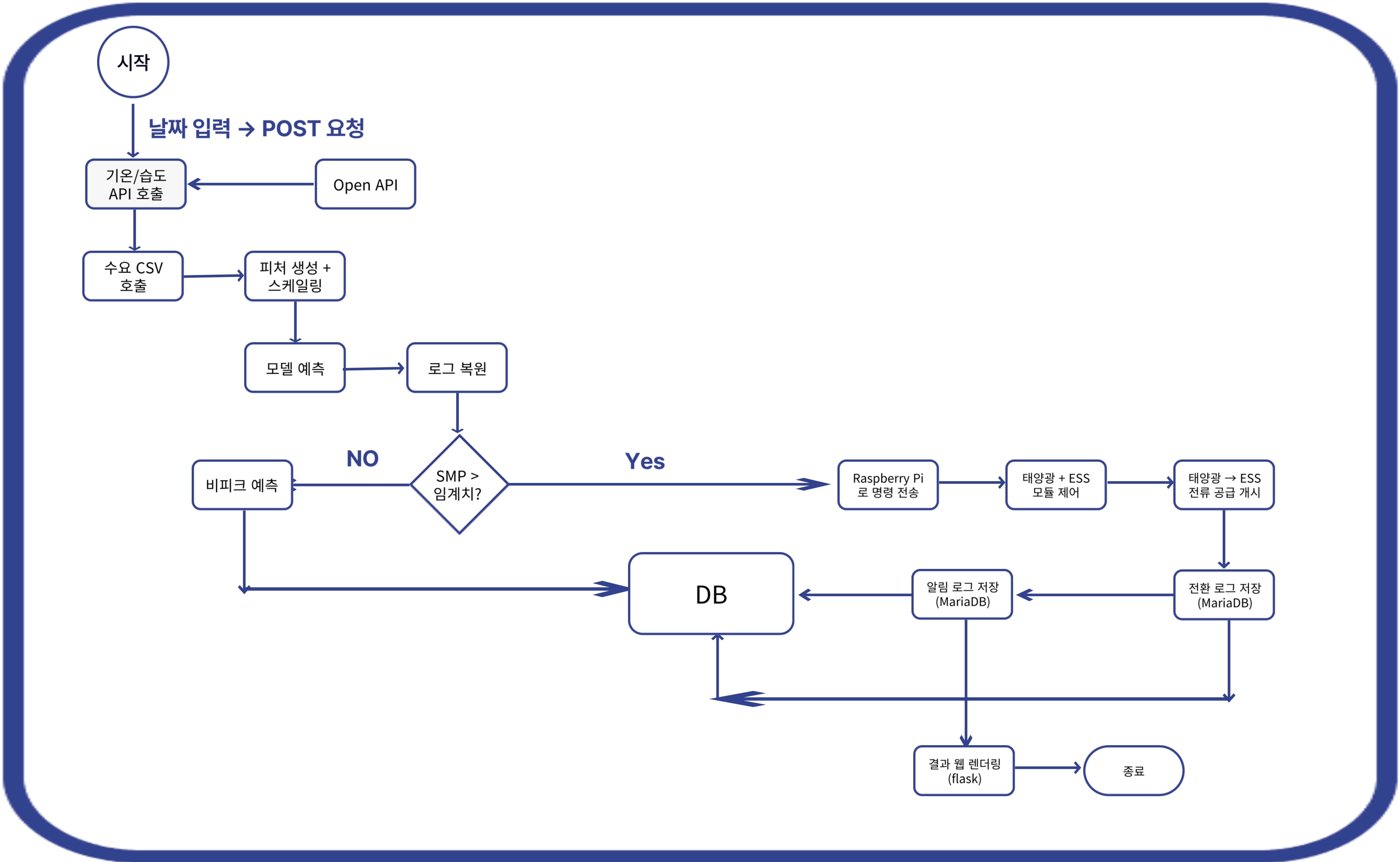
Matplotlib
HTML, CSS
Flask

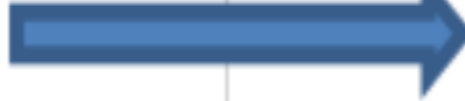
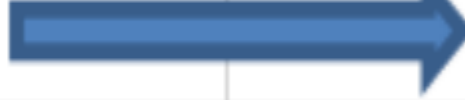
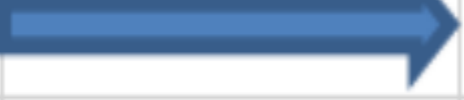
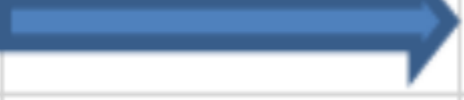
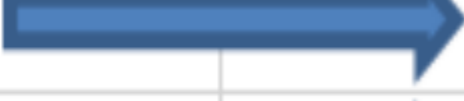
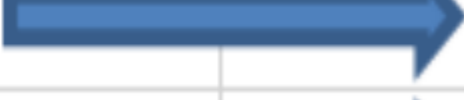
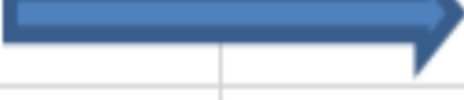
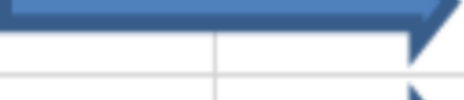
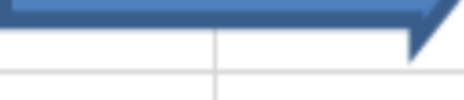
확장성 및 특화 기능

Google Colab
Raspberry Pi 5
MariaDB
Open API (기온·습도 실시간 호출)

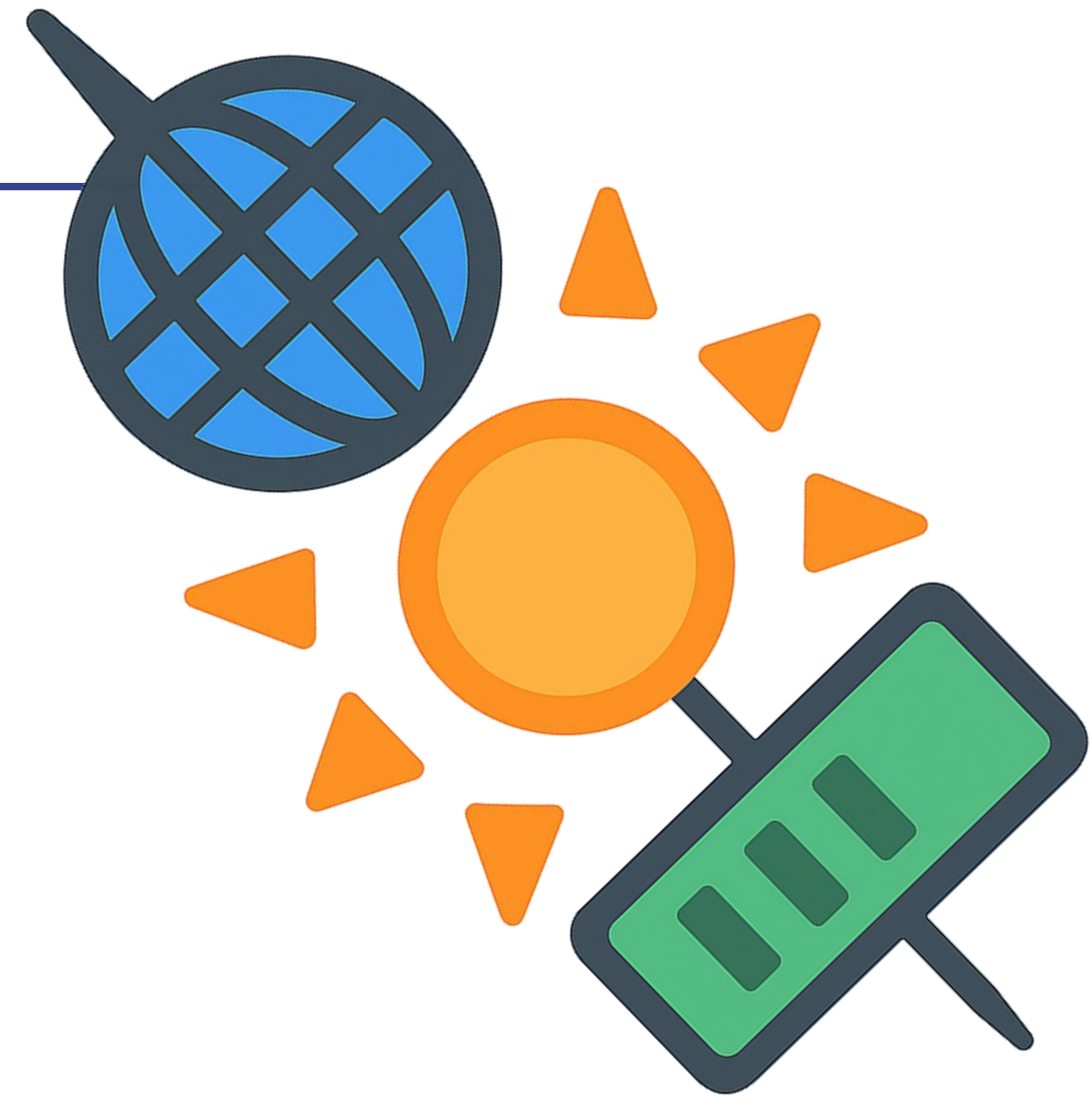
Flow Chart

서론



그룹	작업명	설명	1주차	2주차	3주차	4주차	5주차
기획 및 요구사항 정의	기획 및 요구사항 정의	프로젝트 목적, 시스템 범위, 전환 조건 정의					
	프로젝트 목적 및 범위 지정	전체 시스템의 목표와 경계 설정					
	시스템 구성도 설계	AI + IoT + DB 통합 아키텍처 설계					
	전력 전환 조건 설정	전력 피크 레벨 디자인					
	하드웨어 전압/전류 범위 검토	3V~5V 부하 기준 분석 및 부품 선택					
데이터 수집 및 모델 개발	데이터 수집	시간별SMP, 기온, 습도, 전력 수요 이력 수집					
	데이터 전처리	정규화, 윈도우 생성, 결측치 처리					
	모델 설계	TCN기반 시계열 회귀 모델 설계					
	모델 훈련 및 검증	TensorFlow/Keras로 학습					
	모델 저장 및 배포	.h5 or .pkl 파일로 저장 후 pi에 배포					
하드웨어 구성 및 회로 테스트	회로 설계	릴레이 기반 전환 회로 구성					
	브레드보드 회로 구성	18650, 센서, 부하 연결					
	센서 연결 테스트	INA219 전류/전압 확인					
	릴레이 제어 검증	예측 기반 릴레이 작동 테스트					
Pi 통합 및 Flask 서버 구축	모델 로딩 및 예측 실행	Pi에서 모델 불러오기 및 SMP 예측					
	전환 로직 실행	예측값 기준 자동 릴레이 제어					
	Flask 서버 구축	웹 서버 실행 및 API 라우팅					
	상태 시각화 구현	Chart.js 등으로 예측/상태 표시					
	DB 연동 API 구현	Flask ↔ Maria DB 연결 및 조회 구현					
DB 연동 및 기록	실시간 DB 연동 로직 구현	Pi에서 전환 이벤트 발생 시 자동 DB 저장					
	DB 연동 테스트	실시간 입력 및 조회 테스트					
	로그 기록 검증	실제 예측/상태 로그가 정상적으로 저장되는지 확인					
테스트 및 결과 정리	시나리오 기반 테스트	피크 시간대 전환 시나리오 수행					
	부하 동작 확인	예측결과에 따른 쿨러팬 작동 여부 검증					
	결과 정리	보고서, 로그, 영상 등 문서화 및 자료 정리					

설계



데이터 전처리 과정

설계

Raspberry Pi나 경량 서버 환경에서 운용 가능한 Python 기반 전처리 스크립트를 설계함
시간별 기온, 습도, 전력수요, SMP 데이터를 병합하고 파생변수 및 라벨을 생성하여 예측 모델 입력값으로 가공함

시간 정보 (hour, dayofweek, month)
변화량 (delta_temp, delta_humidity, delta_demand)
이전 값 (prev_SMP, prev_demand)
로그 변환된 SMP (log_SMP)
SMP 기준 피크 여부 (is_peak)
모델 입력용 정렬 및 결측치 처리

preprocessed_smp (주요 컬럼)

컬럼명	타입	설명
time_features	Integer	시간 파생 정보
delta_features	Float	변화량 파생 정보
lag_features	Float	이전 시점 정보
label_and_scaled	Boolean	모델 입력 및 라벨

```
13 # ✅ 날짜 기반 파생변수
14 df["hour"] = df["일시"].dt.hour
15 df["dayofweek"] = df["일시"].dt.dayofweek # 월=0, 일=6
16 df["month"] = df["일시"].dt.month
17
```

날짜를 시간 단위로 분해 (시/요일/월) → 시계열 주기성 반영

```
18 # ✅ 변화량 파생변수
19 df["delta_temp"] = df["기온(* C)"].diff().fillna(0)
20 df["delta_humidity"] = df["습도(%)"].diff().fillna(0)
21 df["delta_demand"] = df["전력수요(MWh)"].diff().fillna(0)
22
```

전 시점 대비 변화량 계산 → 급격한 변화 패턴 감지

```
23 # ✅ 이전 시점 변수
24 df["prev_SMP"] = df["SMP"].shift(1).fillna(method='bfill')
25 df["prev_demand"] = df["전력수요(MWh)"].shift(1).fillna(method='bfill')
26
```

이전 시점 데이터 추가 → 시계열 특성 반영 (지연 변수)

```
27 # ✅ 로그 변환된 SMP (선택)
28 df["log_SMP"] = np.log1p(df["SMP"]) # 모델 입력용
29
```

SMP 로그 변환 → 이상치 영향 줄이고 학습 안정화

```
30 # ✅ 피크 라벨 (선택)
31 df["is_peak"] = (df["SMP"] > 250).astype(int)
32
```

SMP가 250 초과인 경우 피크로 라벨링 → 이진 분류 문제용 라벨 생성

데이터 베이스 설계 과정

설계

Raspberry Pi나 경량 서버 환경에서 가볍고 빠르게 운용 가능한 MariaDB를 선정하였고 Flask + SQLAlchemy + MySQL 기반으로 설계하였다. Python + SQLAlchemy에서 처리한 데이터 결과를 쉽게 확인하고 수정 가능한 HeidiSQL을 사용하여 작업성을 높임

어떤 데이터가 필요하고, 어떤 기능을 구현해야 하는지 결정:
전환 시각 (언제 태양광으로 바뀌었는가)
전환 이유 (SMP 급등, 전력 피크 예측 등)
현재 전력원 (태양광 / 일반 전력)
축전지 잔여량
일조량 예보 기반 예상 비축량
기계학습 예측 결과 (피크 예측치)

테이블 구조는 아래와 같이 구성함

solar_switch_log (태양광 전환이력)		
컬럼명	타입	설명
id	Integer(PK)	고유식별자
switched_at	DateTime	전환 발생 시각
trigger_reason	String	전환 트리거 이유

```
USE energy_system;

GRANT ALL PRIVILEGES ON energy_system.* TO 'energy_user'@'localhost';
FLUSH PRIVILEGES;
-- CREATE USER 'energy_user'@'localhost' IDENTIFIED BY 'supersecret123!';
-- CREATE DATABASE energy_system;
```

SQLAlchemy을 사용하여 테이블을 생성하기 이전에 권한을 설정함

```
app = Flask(__name__)
app.config['SECRET_KEY'] = 'secret!'
app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql+pymysql://energy_user:supersecret123!@localhost/energy_system'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

socketio = SocketIO(app, async_mode='threading')
db = SQLAlchemy(app)
```

MariaDB와 Flask를 연결

```
class SolarSwitchLog(db.Model):
    __tablename__ = 'solar_switch_log'
    id = db.Column(db.Integer, primary_key=True)
    switched_at = db.Column(db.DateTime, nullable=False)
    trigger_reason = db.Column(db.String(255))

    with app.app_context():
        db.create_all()
```

Python + SQLAlchemy를 이용하여 테이블을 만듦

```
@app.route('/switch')
def trigger_solar_switch():
    now = datetime.now()
    now_str = now.strftime('%H:%M:%S')
    socketio.emit('realtime_data', {'time': now_str, 'source': 'solar'})

    log = SolarSwitchLog(switched_at=now, trigger_reason='테스트 전환')
    db.session.add(log)
    db.session.commit()

    return f'태양광 으로 전환됨 ({now_str})'
```

브라우저 혹은 API 요청에따라 태양광 비축에너지로 전환된 기록을 DB에 전송하도록 설정

모델 설계 과정

설계

경량 서버 환경에서도 작동 가능한 Conv1D-LSTM 기반 전력 예측 모델을 TensorFlow로 구현
전처리된 시계열 데이터를 학습하여 SMP 급등 여부를 예측하고, 이를 ESS 또는 태양광 전환 조건으로 활용할 수 있도록 설계함

시계열 특징 추출 (시간, 요일, 변화율 등)
SMP 예측 (로그 변환 → 역변환)
급등 예측 여부 판단 ($\Delta 15W$ 이상)
미래 SMP 평균 및 최대값 기반 피크 감지
학습 스케일러 및 피쳐 저장

모델 주요 기능

항목명	설명	적용 목적
extract_time_pattern()	시간 흐름 학습	예측 정밀도 향상
detect_spike_zone()	SMP 변화 감지	사전 경고 판단
combine_multi_features()	다변수 입력	현실성 반영
generate_switch_signal()	임계치 판단	자동 제어 연동

```
# ✅ 모델 정의 (Conv1D + LSTM 하이브리드 구조)
model = Sequential([
    Input(shape=(X_scaled.shape[1], 1)),
    Conv1D(64, kernel_size=3, padding='causal', activation='relu'),
    BatchNormalization(),
    Dropout(0.2),
    LSTM(64, return_sequences=True),
    LSTM(32),
    Dense(64, activation='relu'),
    Dense(1)
])
```

Conv1D와 LSTM을 결합해 시계열 패턴을
효과적으로 학습하는 모델 구조를 정의함

```
df["delta_prev_smp"] = df["prev_SMP"] - df["prev_SMP_t2"]
df["rolling_smp"] = df["prev_SMP"].rolling(window=3).mean()
df["rate_prev_smp"] = df["delta_prev_smp"] / (df["prev_SMP_t2"] + 1e-5)
```

SMP의 변화량, 비율, 이동평균 등을 계산해 모델 입력용 피쳐를 생성함

```
# ✅ 모델 학습
print("\n🔥 모델 학습 시작...")
history = model.fit(
    X_scaled, y,
    epochs=200,
    batch_size=32,
    validation_split=0.15,
    callbacks=[early_stop],
    verbose=2
)

# ✅ 저장
os.makedirs(save_dir, exist_ok=True)
model.save(os.path.join(save_dir, "model.keras"))
joblib.dump(scaler, os.path.join(save_dir, "scaler.pkl"))
joblib.dump(features, os.path.join(save_dir, "features.pkl"))
print("✅ 모델 및 스케일러 저장 완료!")
```

모델을 학습하고, 학습된 모델과 스케일러,
\피쳐 구성을 파일로 저장하여 배포 또는 재사용에 대비함

웹 설계 과정

설계

교육과정에서 배운 Flask를 이용하여 웹서버를 설계하고 HTML, CSS, JavaScript를 이용하여 구성하였으며, 실시간 데이터를 활용할 예정인만큼 실시간 양방향 통신이 가능한 SocketIO를 이용하였고, 매일 일조량 예보 정보를 사용할것이기에 datetime라이브러리를 이용함

기능적 필요사항 요점:

실시간 SMP(전력도매가) 예측 결과를 시각화
태양광 전환 여부 기록 및 기상청 API를 통한 기온·습도 수집
일조량 기반 태양광 발전량 예측 포함
SocketIO를 통해 실시간 전환 이벤트를 알림

페이지는 아래와 같이 구성함

ESS잔여량
현재 전력원
태양광 전환 이력
피크 예측 결과
LED

```
app = Flask(__name__)
app.config['SECRET_KEY'] = 'secret!'
app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql+pymysql://energy_user:supersecret123!@localhost/energy_system'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

socketio = SocketIO(app, async_mode='threading')
db = SQLAlchemy(app)
```

Flask 프레임워크를 사용하여 웹서버를 초기화하고,
SocketIO를 통해 실시간 통신 기능 설정

```
@socketio.on('connect')
def on_connect():
    print('클라이언트 연결됨')

# ----- 태양광 전환 테스트 -----
<< /switch
@app.route('/switch')
def trigger_solar_switch():
    now = datetime.now()
    now_str = now.strftime('%H:%M:%S')
    socketio.emit('realtime_data', *args={'time': now_str, 'source': 'solar'})
    log = SolarSwitchLog(switched_at=now, trigger_reason='테스트 전환')
    db.session.add(log)
    db.session.commit()
    return f'태양광 으로 전환됨 ({now_str})'
```

/switch 엔드포인트 호출 시 실시간으로 SocketIO를 통해 프론트에
태양광 전환 이벤트를 알림

```
socket.on('realtime_data', (data) => {
    document.getElementById("power_source").innerText = data.source === 'solar' ? "태양광" : "일반 전력";
});
```

클라이언트에서는 Socket을 통해 수신된 데이터로 전력원 변경 상황을
실시간 표시한다.

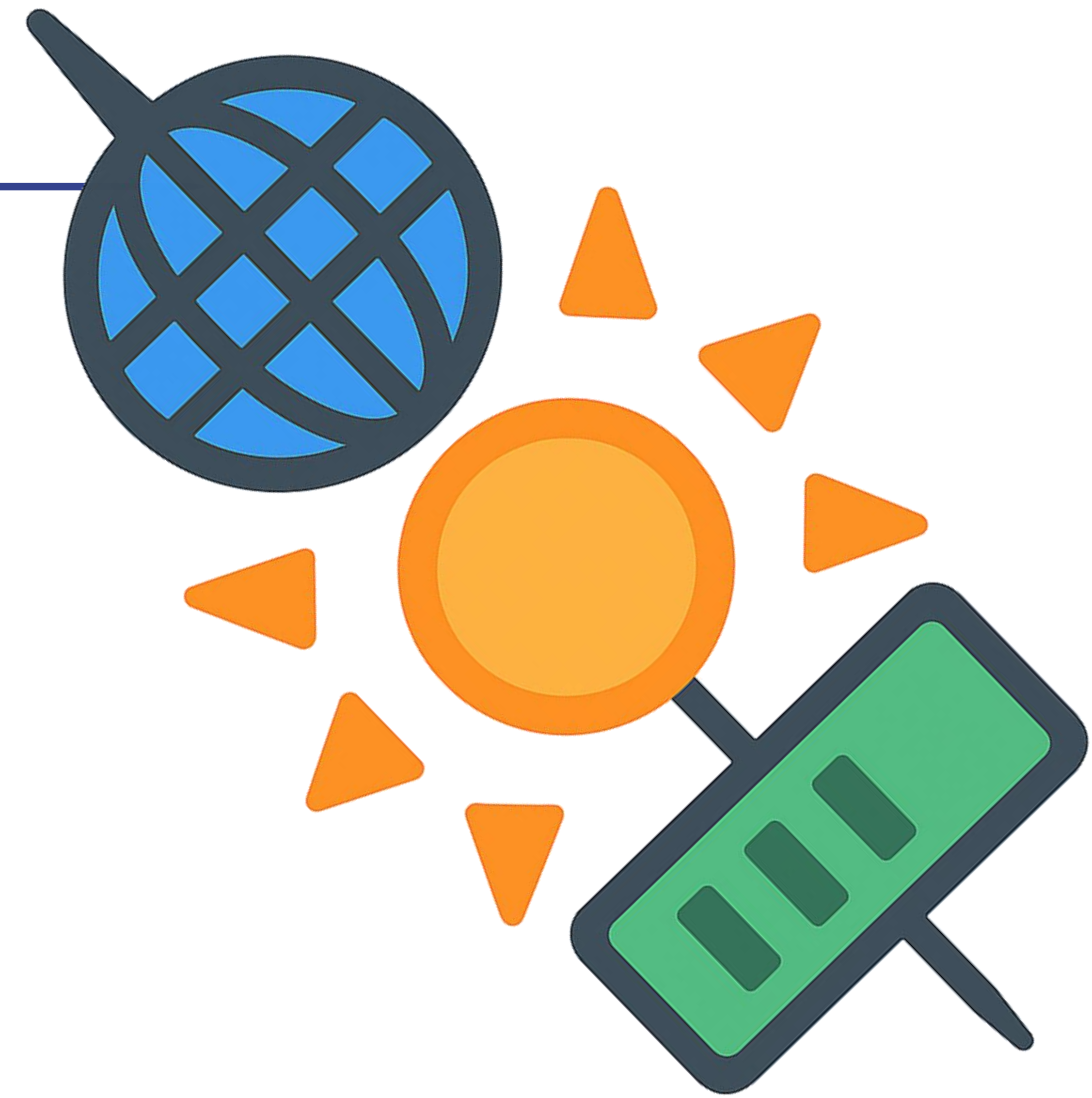
```
res = requests.get(
    url: 'https://apihub.kma.go.kr/api/typ01/url/kma_sfctm2.php',
```

기상청 일사량 데이터 API를 호출하고 응답 받은 HTML/XML
데이터를 테이블 데이터를 쉽게 추출하기 위해 BeautifulSoup으로 파싱

```
radiation_mj = float(cols[11].text.strip())
sunshine = float(cols[12].text.strip())
estimated_kwh = round(radiation_mj * 0.15, 2)
```

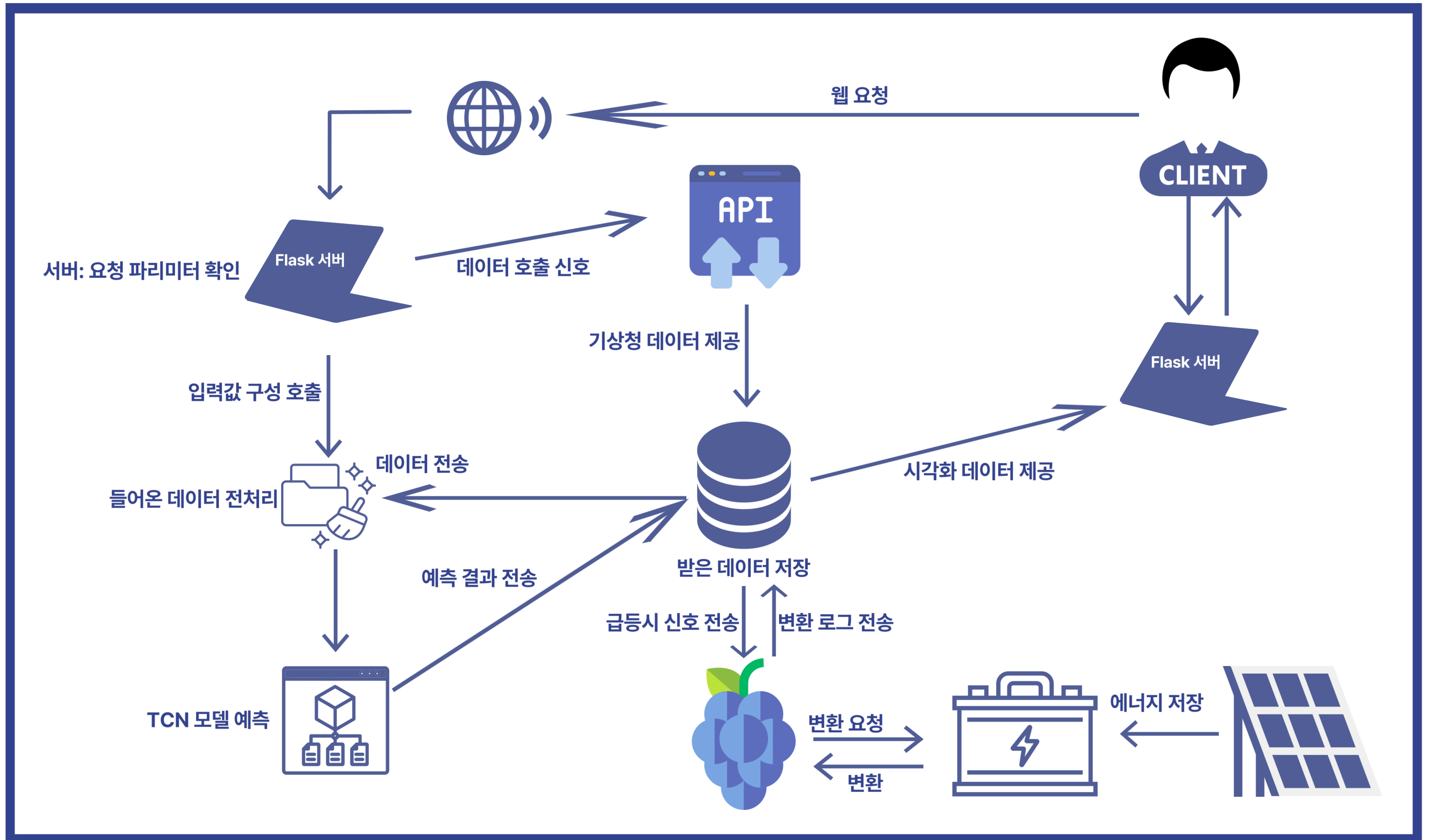
12번째 열(인덱스 11)에 있는 일사량 값(MJ/m²)을 숫자로변환해서
저장하고 일사량(MJ/m²)을 예상 발전량(kWh)으로 환산함

본론



시스템 구성도

본론




```

ex.py x
1 from flask import Flask, request
2 import RPi.GPIO as GPIO
3 import atexit
4
5 app = Flask(__name__)
6
7 # ? GPIO pin assignments
8 SAFE_LED = 23      # Green - SAFE
9 WARNING_LED = 24   # Yellow - WARNING
10 DANGER_LED = 25    # Red - DANGER
11
12 # ? GPIO setup
13 GPIO.setmode(GPIO.BCM)
14 GPIO.setup(SAFE_LED, GPIO.OUT)
15 GPIO.setup(WARNING_LED, GPIO.OUT)
16 GPIO.setup(DANGER_LED, GPIO.OUT)
17
18 # Turn all LEDs off at start
19 GPIO.output(SAFE_LED, GPIO.LOW)
20 GPIO.output(WARNING_LED, GPIO.LOW)
21 GPIO.output(DANGER_LED, GPIO.LOW)
22
23 # ? Cleanup on exit
24 atexit.register(GPIO.cleanup)
25
26 # ? Root route
27 @app.route("/")
28 def index():
29     return "Flask LED Control Server is running."
30
31 # ? LED control route
32 @app.route("/led", methods=["POST"])
33 def control_led():
34     data = request.json
35     action = data.get("action")
36
37     # Turn off all LEDs first
38     GPIO.output(SAFE_LED, GPIO.LOW)
39     GPIO.output(WARNING_LED, GPIO.LOW)
40     GPIO.output(DANGER_LED, GPIO.LOW)
41

```

```

# LED 판단 및 전송
try:
    top2 = df_test["예측"].nlargest(2)
    avg_top2 = float(top2.mean())

    if avg_top2 >= 220:
        led_action = "danger_on"
    elif avg_top2 >= 170:
        led_action = "warning_on"
    else:
        led_action = "safe_on"

    pi_url = "http://192.168.1.16:5000/led"
    res = requests.post(pi_url, json={"action": led_action})
    print(f"LED sent: {led_action} | avg_top2 = {avg_top2:.2f} | status = {res.status_code}")

except Exception as e:
    print(f"LED send failed: {e}")
    led_action = "error"
    avg_top2 = None

results = []
for _, row in df_test.iterrows():
    results.append({
        "time": row["일시"].strftime("%H:%M"),
        "predicted": round(float(row["예측"]), 2),
        "actual": round(float(row["실제"]), 2),
        "note": "급등" if row["급등"] else ""
    })

return jsonify({
    "results": results,
    "r2": r2,
    "avg_top2_smp": round(avg_top2, 2) if avg_top2 else None,
    "led_signal": led_action
})

except Exception as e:
    return jsonify({'error': str(e)}), 500

```

클라이언트(예측 서버)로부터 POST 요청으로 action 값을 전달받아 그에 따라 라즈베리파이에 연결된 LED(SAFE/WARNING/DANGER)를 실제로 제어하는 하드웨어 전용 서버 코드이며 서버 종료 시 GPIO를 정리해 안전하게 종료됨

머신러닝 모델의 예측 결과를 바탕으로 급등 조건을 판단하고, LED 상태를 결정하여 하드웨어 LED 서버로 POST 전송하며 동시에 클라이언트 웹에 전송할 예측 결과 리스트도 생성하여 JSON 응답으로 반환함


```
document.getElementById("datePicker").addEventListener("change", () => {
  const date = document.getElementById("datePicker").value;
  if (!date) return;
  fetch(`/api/ml_predict?date=${date}`)
    .then(res => res.json())
    .then(data => {
      const list = document.getElementById("prediction_result");
      list.innerHTML = "";

      if (data.error) {
        console.error("서버에서 받은 에러:", data.error);
        list.innerHTML = `<li>❌ 서버 오류: ${data.error}</li>`;
        return;
      }

      data.results.forEach(item => {
        const li = document.createElement("li");
        li.innerHTML = `${item.time} - 예측: ${item.predicted}#, 실제: ${item.actual}# ${item.note}`;
        list.appendChild(li);
      });

      const r2 = document.createElement("li");
      r2.innerHTML = `☑ 설명력 (R²): ${data.r2}`;
      list.appendChild(r2);

      // ✅ LED 상태 표시
      const ledText = document.getElementById("led_status");
      if (data.led_signal === "danger_on") {
        ledText.innerHTML = "🔴 위험 (DANGER)";
        ledText.style.color = "red";
      } else if (data.led_signal === "warning_on") {
        ledText.innerHTML = "🟡 경고 (WARNING)";
        ledText.style.color = "orange";
      } else if (data.led_signal === "safe_on") {
        ledText.innerHTML = "🟢 안전 (SAFE)";
        ledText.style.color = "green";
      } else {
        ledText.innerHTML = "LED 상태 알 수 없음";
        ledText.style.color = "gray";
      }
    }).catch(err => {
      console.error("예측 fetch 실패:", err);
      document.getElementById("prediction_result").innerHTML = `<li>❌ 예측 fetch 실패</li>`;
    });
});
```

사용자가 날짜를 선택하면 서버(/api/ml_predict)에 예측 요청을 보고, 응답으로 받은 시간별 SMP 예측 결과와 LED 신호(위험/경고/안전)를 웹 UI에 시각적으로 출력하는 클라이언트 측 자바스크립트 코드로 에러 발생 시 사용자에게 오류 메시지도 함께 출력함

```
<script>
  const socket = io();

  socket.on('connect', () => {
    console.log("서버에 연결됨");
  });

  socket.on('realtime_data', (data) => {
    document.getElementById("power_source").innerHTML = data.source === 'solar' ? "태양광" : "일반전력";

    const logElement = document.getElementById("switch_log");
    const newLog = document.createElement("li");
    newLog.innerHTML = `태양광으로 전환됨! 시간: ${data.time}`;
    logElement.prepend(newLog);
  });

  function loadPeakRisk() {
    fetch('/api/predicted_peak')
      .then(res => res.json())
      .then(data => {
        const peak = data.peak_risk_index.toFixed(2);
        const isPeak = data.is_predicted_peak ? '⚠ 위험' : '정상';
        document.getElementById('peak_risk').innerHTML = `${peak} (${isPeak})`;
      }).catch(err => {
        document.getElementById('peak_risk').innerHTML = '데이터 로딩 실패';
        console.error(err);
      });
  }

  function loadSolarEstimate() {
    fetch('/api/solar_estimate')
      .then(res => res.json())
      .then(data => {
        const est = data.solar_estimate_kwh;
        const raw = data.raw_radiation;
        document.getElementById('solar_estimate').innerHTML = `
          예상 발전량: ${est} kWh (예보 일사량: ${raw} MJ/m²)
        `;
      }).catch(err => {
        document.getElementById('solar_estimate').innerHTML = '데이터 로딩 실패';
        console.error(err);
      });
  }
}
```

서버에서 받은 실시간 전력 전환 정보와 예측 지표(API 응답)를 받아 현재 전력원, 전환 로그, 전력 피크 위험도를 웹 화면에 표시하는 클라이언트 코드

데이터베이스

본론

```
mk1 mk1.sql
1 USE energy_system;
2
3 GRANT ALL PRIVILEGES ON energy_system.* TO 'energy_user'@'localhost';
4 FLUSH PRIVILEGES;
5 -- CREATE USER 'energy_user'@'localhost' IDENTIFIED BY 'supersecret123!';
6 -- CREATE DATABASE energy_system;
7 GRANT SELECT, INSERT, UPDATE, DELETE
8 ON energy_system.* TO 'energy_user'@'localhost';
9 FLUSH PRIVILEGES;
10
11 -- CREATE DATABASE IF NOT EXISTS energy_system;
12
13 CREATE TABLE IF NOT EXISTS smp_status (
14   id INT AUTO_INCREMENT PRIMARY KEY,
15   timestamp DATETIME NOT NULL,
16   peak_risk_index FLOAT,
17   is_predicted_peak BOOLEAN
18 );
19
20 CREATE TABLE IF NOT EXISTS energy_status (
21   id INT AUTO_INCREMENT PRIMARY KEY,
22   timestamp DATETIME NOT NULL,
23   power_source ENUM('grid', 'solar') NOT NULL,
24   battery_level FLOAT,
25   solar_estimate_kwh FLOAT
26 );
27
28 CREATE TABLE IF NOT EXISTS solar_switch_log (
29   id INT AUTO_INCREMENT PRIMARY KEY,
30   switched_at DATETIME NOT NULL,
31   trigger_reason VARCHAR(255)
32 );
33
34 SELECT * FROM solar_switch_log ORDER BY id DESC;
35 SHOW TABLES;
```

solar_switch_log (15r × 3c) TABLE_NAMES (5r × 1c)

#	Tables_in_energy_system
1	energy_status
2	predicted_peak
3	smp_status
4	solar_estimate
5	solar_switch_log

sql스크립트에서 직접 작성한 테이블과
SQLAlchemy Python으로 작성한 테이블들이
잘 생성된것을 확인

```
class SolarSwitchLog(db.Model): 1개의 사용 위치
    __tablename__ = 'solar_switch_log'
    id = db.Column(db.Integer, primary_key=True)
    switched_at = db.Column(db.DateTime, nullable=False)
    trigger_reason = db.Column(db.String(255))

class SolarEstimate(db.Model):
    __tablename__ = 'solar_estimate'
    id = db.Column(db.Integer, primary_key=True)
    date = db.Column(db.Date, unique=True, nullable=False)
    radiation_mj = db.Column(db.Float, nullable=False)
    estimated_kwh = db.Column(db.Float, nullable=False)
    sunshine = db.Column(db.Float, nullable=True)

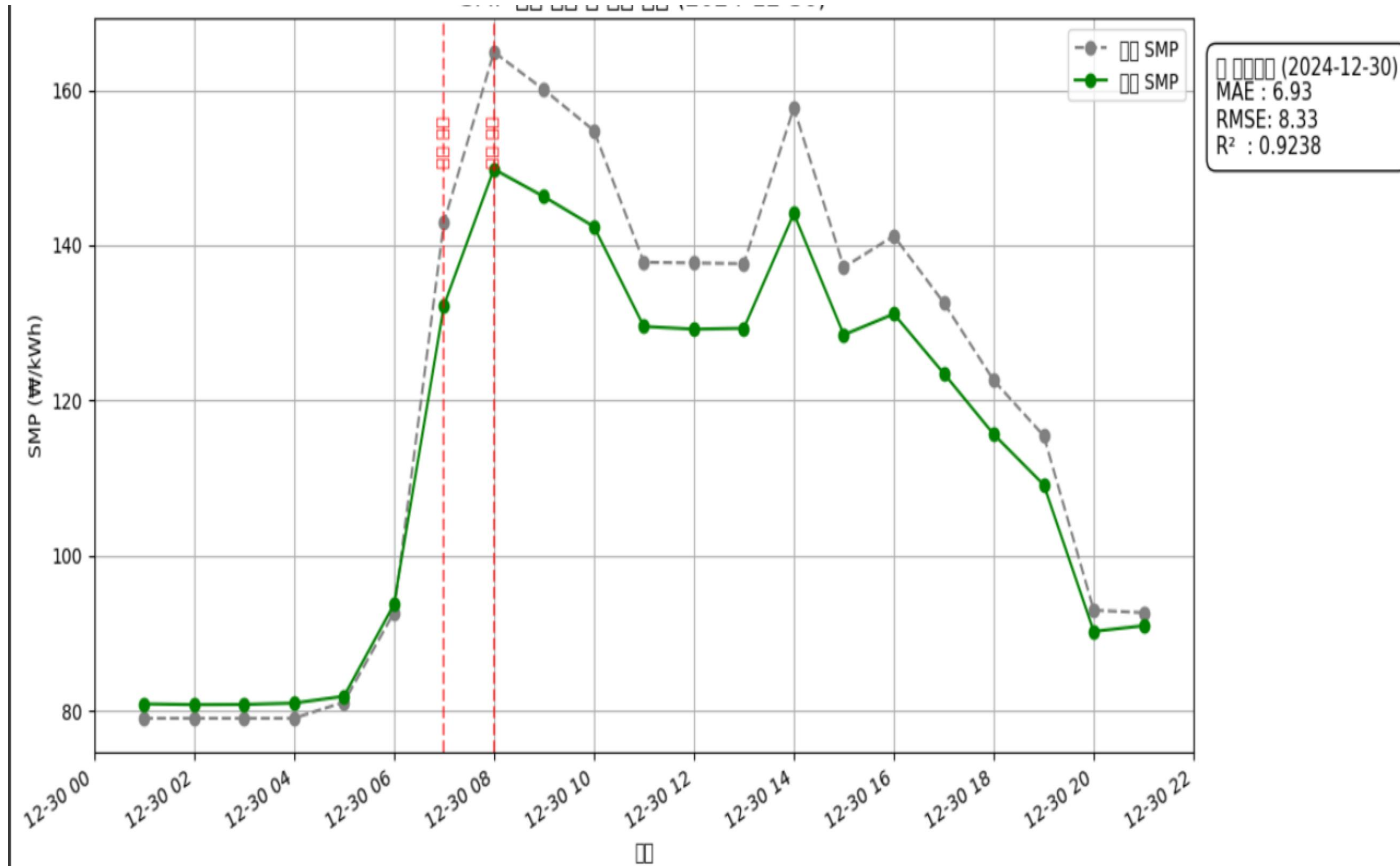
with app.app_context():
    db.create_all()
```

solar_switch_log (15r × 3c)					TABLE_NAMES (5r × 1c)				
#	id		switched_at	trigger_reason					
1	15		2025-06-23 12:59:35	테스트 전환					
2	14		2025-06-23 12:58:49	테스트 전환					
3	13		2025-06-23 11:37:19	수동 테스트 전환					
4	12		2025-06-23 11:37:00	수동 테스트 전환					
5	11		2025-06-23 11:36:45	수동 테스트 전환					
6	10		2025-06-23 11:32:50	테스트 전환					

태양광 변환 시각과 변환 원인 또한 정상적으로 DB에 기록됨

머신러닝 모델 결과

본론



4. 예상 태양광 비축량

(일조량 예보 API 연동 계산값)

예상 발전량: 0.61 kWh (예보 일사량: 12.13 MJ/m²)

5. 선택 날짜 전력 예측 결과

(날짜를 선택하면 해당 날짜의 시간별 예측 결과가 표시됨)

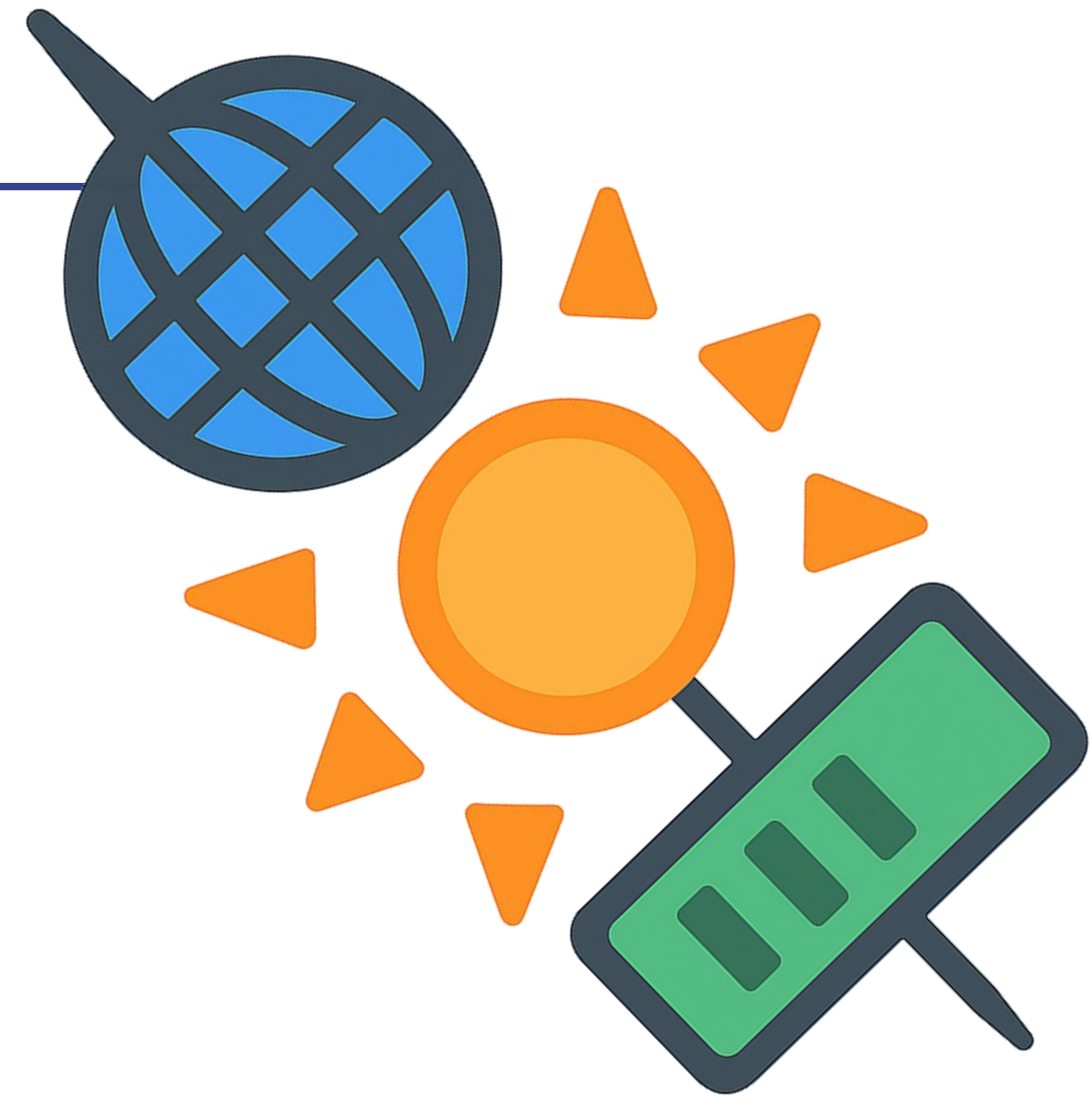
2024-12-11

- 00:00 - 예측: 83.11₩, 실제: 83.43₩
 - 01:00 - 예측: 83.95₩, 실제: 82.1₩
 - 02:00 - 예측: 82.77₩, 실제: 82.1₩
 - 03:00 - 예측: 91.15₩, 실제: 82.1₩
 - 04:00 - 예측: 109.36₩, 실제: 91.46₩ 급등
 - 05:00 - 예측: 123.9₩, 실제: 106.25₩
 - 06:00 - 예측: 150.62₩, 실제: 121.62₩ 급등
 - 07:00 - 예측: 158.66₩, 실제: 153.64₩
 - 08:00 - 예측: 157.12₩, 실제: 156.04₩
 - 09:00 - 예측: 155.06₩, 실제: 159.67₩
 - 10:00 - 예측: 140.74₩, 실제: 153.64₩
 - 11:00 - 예측: 147.65₩, 실제: 137.97₩
 - 12:00 - 예측: 149.32₩, 실제: 143.06₩
 - 13:00 - 예측: 160.66₩, 실제: 143.07₩
 - 14:00 - 예측: 153.43₩, 실제: 158.57₩
 - 15:00 - 예측: 151.16₩, 실제: 153.64₩
 - 16:00 - 예측: 145.91₩, 실제: 143.4₩
 - 17:00 - 예측: 143.57₩, 실제: 143.41₩
 - 18:00 - 예측: 136.7₩, 실제: 137₩
 - 19:00 - 예측: 115.08₩, 실제: 136.09₩
 - 20:00 - 예측: 121.66₩, 실제: 112.28₩
 - 21:00 - 예측: 121.91₩, 실제: 120.02₩
 - 22:00 - 예측: 108.12₩, 실제: 118.32₩
 - 23:00 - 예측: 93.05₩, 실제: 102.61₩
- 설명력 (R²): 0.839

급등구간 예측에 성공하였으며 예측정확도는 92%를 기록함

사전 예측가능한 모델을 구현

결론



1

스케일링 방식의 부적절함

문제점: MinMaxScaler나 StandardScaler 사용 시,
SMP 급등구간이 너무 평탄화됨
해결: 이상치에 강한 RobustScaler 사용 (중앙값 + IQR 기반)

2

예측 시점 타이밍 불일치

문제점: log_SMP.shift(-2)을 사용하지 않고 현재 시점
예측값으로 학습 → 2시간 후 SMP 예측이 안됨
해결: 반드시 shift(-2) 적용해서 2시간 후 값을 타겟으로 설정함

3

예측 성능 문제

문제점: 기온·습도만으로는 예측 성능이 낮았음
해결: 전력수요, 시간 정보, 과거 SMP값을 포함한
다변량 모델로 개선

4

모델 선택 문제 (LSTM vs TCN)

문제점: LSTM은 느리고 정규화 없는 불안정
해결: 빠르고 안정적인 TCN(Causal Conv1D) 구조로 전환

5

파생변수 너무 많아짐

문제: 피처가 많아져 과적합 우려
해결: 성능에 기여하는 피처만 선별, 불필요한 항목 제거

6

모델의 직렬구조 오류

문제: y값 누락으로 예측 결과 비교 불가
해결: 루프 내 y_test 지정 → 예측값과 실제값 함께 저장

1

기술 고도화

- 태양광 장비 설치 및 시연
- SMP 예측 모델 고도화
- 실시간 대시보드 및 ESS 전송 최적화

2

수익 모델 구체

- 구독 이탈 방지 및 리포트 판매
- 지자체·한전 연계 공공사업 제안
- 태양광·ESS 기반 전력 플랫폼화

3

제도 및 정책 대응

- 에너지바우처 연계: 취약계층 무상/부분 지원
- 전력 역송 협의: ESS 초과 전력 관련 한전·지자체 조율

4

ESG 인증 및 투자 유치

- ESG 인증 추진: 탄소절감 리포트 + 자동화 시스템
- 투자 유치: ESG 펀드·임팩트 투자자 대상
- 해외 보급 검토: 전력불안정 국가 대상 경량 모델 개발

5

추가 아이디어

- 마이크로그리드 구축: 마을 단위 전력망 구성
- 개인 전력 거래화: 앱 기반 P2P 태양광 판매 플랫폼

6

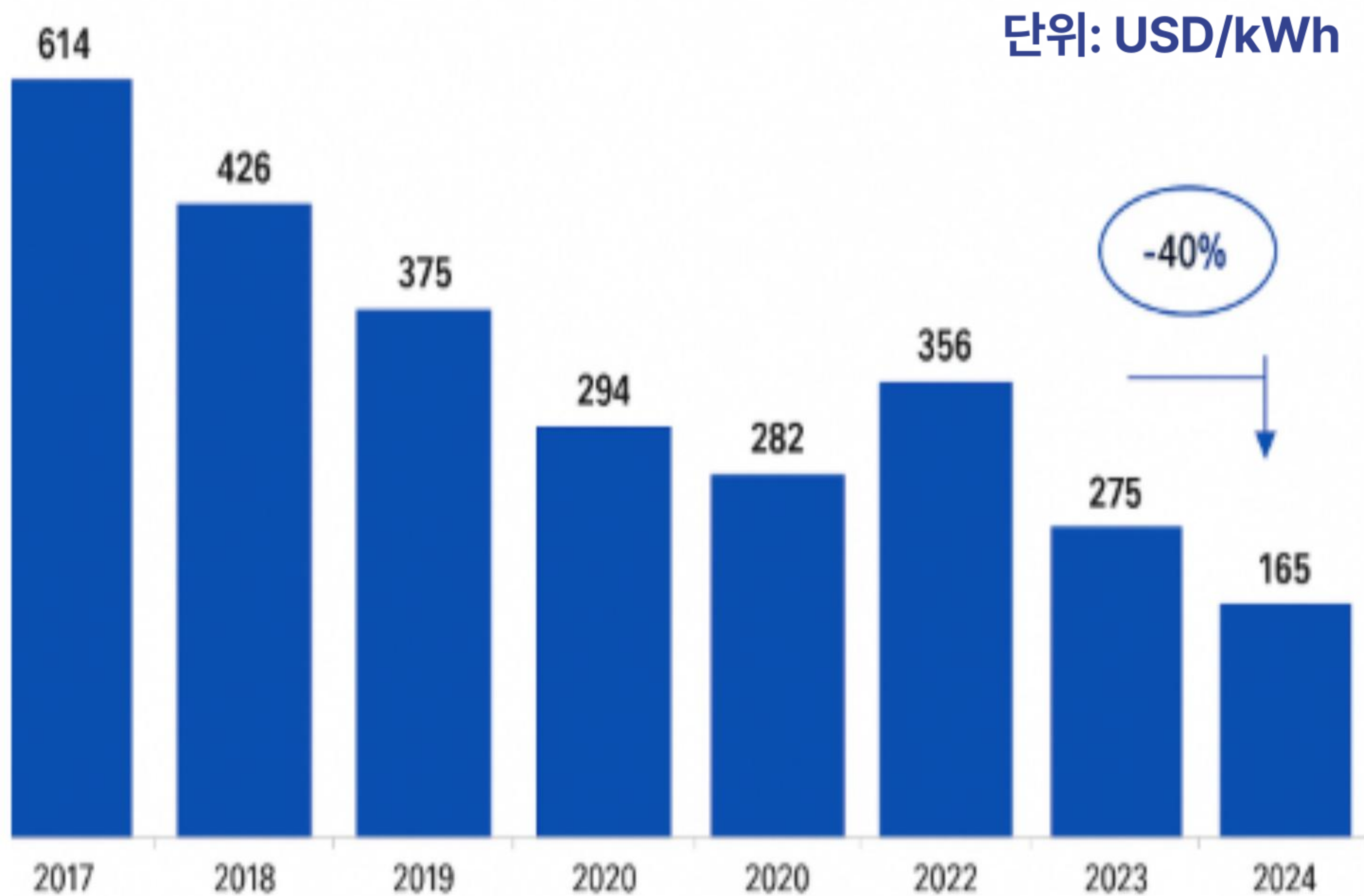
보안 및 인증 체계 강화

- 데이터 위·변조 방지 로직 도입
- 정부/공공사업 대응을 위한 보안 인증 준비

미래의 완자 IoT 소개

결론

ESS 비용 현실화



자료5 출처: BloombergNEF

태양광으로 희망을 되찾은 마을



자료6 출처: Knowable magazine 나이지리아 Mbiabet Esieyere 마을

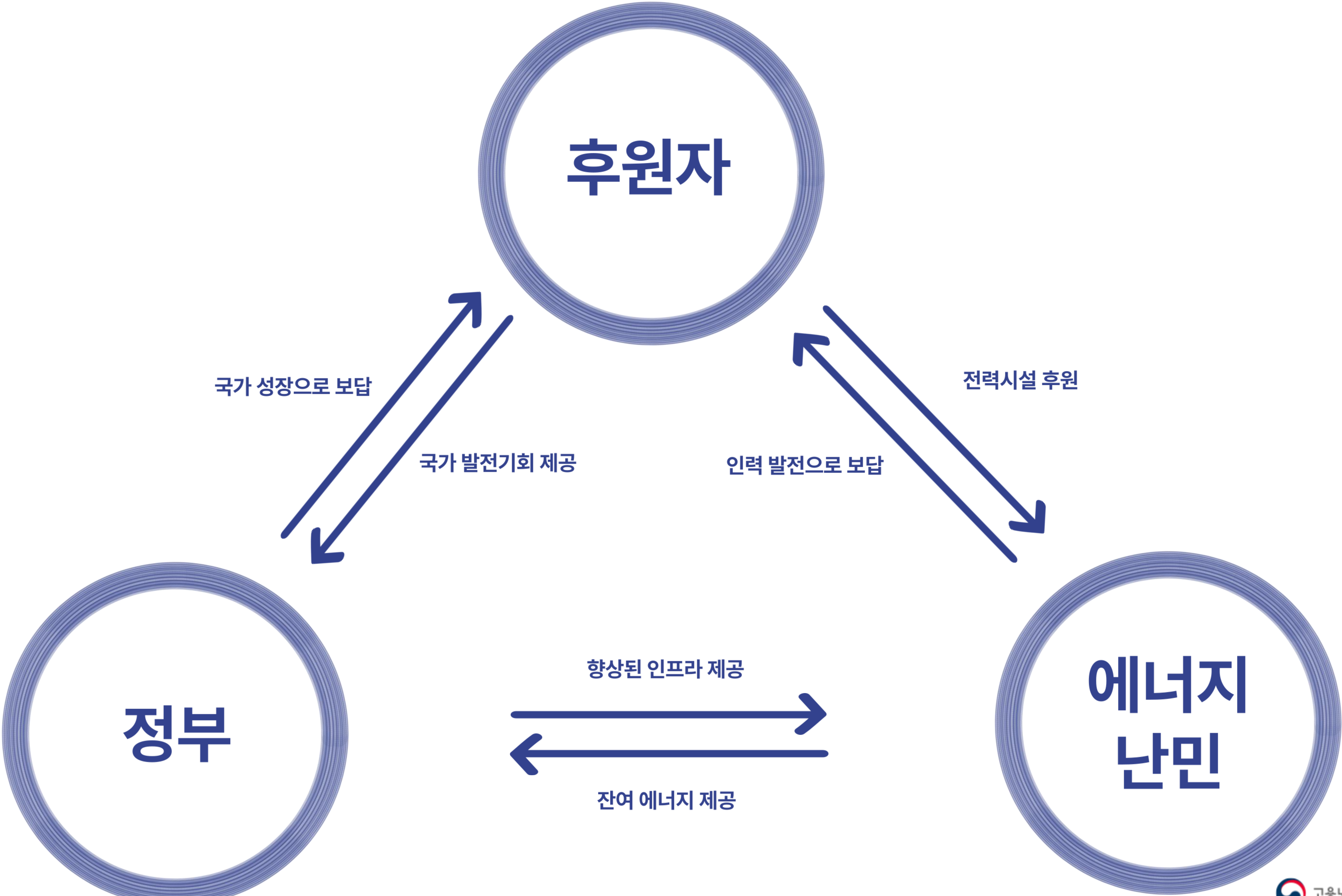
- ESS(에너지 저장 장치) 설치 단가는 2017년 대비 약 73% 하락, 2024년 기준 165 USD/kWh 까지 하락

- 비용 감소는 ESS 도입 확산과 친환경 에너지 전환 가속화의 핵심 요인

- 나이지리아 Mbiabet Esieyere 마을, 2022년 솔라 미니그리드 구축으로 자립형 전력망 확보

- 태양광과 ESS 연계로 외부 전력망 없이도 안정적인 전력 공급 가능

완자 IoT희망전력사업





지속가능한 에너지 솔루션

- 전력 피크 예측 정확도 향상
- 태양광 에너지 활용 최적화
- 에너지 효율성 20% 이상 증대
 - 스마트 그리드 기술 선도
 - 에너지 자립도 향상 지원
- 미래 에너지 시장 혁신 주도

경제적 가치 창출

- 가구당 연간 전기요금 15% 절감
- 에너지 시장 새로운 비즈니스 모델
 - 1000가구 목표, 12억 원 예상 수익
- 국가 에너지 비용 절감에 기여
 - 신규 일자리 창출 효과
- 에너지 산업 경쟁력 강화

환경 보호 기여

- 연간 CO2 배출량 10% 감소
- 재생에너지 사용 비중 확대
 - 에너지 절약 문화 확산
 - 자원 효율적 사용 촉진
- 환경 친화적 기술 발전
- 탄소중립 목표 달성 지원

사회적 영향력

- 에너지 복지 실현
- 지역 사회 에너지 자립 지원
 - 에너지 시민 의식 고취
- 지속가능한 도시 발전 기여
 - 에너지 민주화 촉진
- ESG 가치 실현 선도

1

이제우

실물 모델이 마련되지 않아 아쉬움이 많이 남지만
실물 모델이 없는대신 그만큼 프로젝트 구조에대해
깊게 탐구해볼수있는 기회였다 평소 기부에도 많은
관심이 있는만큼 이후에 이 프로젝트를 발판삼아
더욱 발전시켜 자그마한 모델로 시작해서
미래에는 마을단위를 구성할수있는
완자IOT 호프타운을 만들어보고싶은 생각이 들었다.

2

김강수

짧은시간이었지만 급한만큼
많은 시행착오를 거쳐 프로젝트 시작 전과 달리
머신러닝에대해 개인적으로 비약적인 성장을
하였다고 스스로 느끼고있음에 뿌듯했다.
다만 이것이 자만이 아닌 한걸음의 성취임을 알기에
더욱 정진하여 어쩌면 미래에 많은이에게 희망을
안겨줄수있는 완자IOT 호프타운 건설에
적극 참여하고싶은 느낌을 받았다.

1. 주요 통계 및 글로벌 리포트 출처

IEA – 세계 전력 소비 개요(자료1)

<https://www.iea.org/reports/electricity-information-overview/electricity-consumption>

IRENA – World Energy Transition Outlook 2024(자료2)

https://www.irena.org/-/media/Files/IRENA/Agency/Publication/2024/Nov/IRENA_World_energy_transitions_outlook_2024_Summary.pdf

BloombergNEF – 2025 에너지 전환 투자 보고서(자료3)

<https://about.bnef.com/insights/finance/global-investment-in-the-energy-transition-exceeded-2-trillion-for-the-first-time-in-2024-according-to-bloombergnef-report>

IEA – Renewables 2024(자료4)

<https://www.iea.org/reports/renewables-2024>

BloombergNEF – 에너지 저장 비용 하락 관련(자료5)

<https://about.bnef.com>

2. 공공데이터 및 통계 플랫폼

공공데이터포털

<https://www.data.go.kr/>

기상청 기상자료개방포털

<https://data.kma.go.kr/data/grnd/selectAsosRltmList.do?pgmNo=36>

한국전력거래소

<https://www.kpx.or.kr/>

전력통계정보시스템 (SMP 등 확인 가능)

<https://epsis.kpx.or.kr/epsisnew/selectEkmaSmpShdChart.do?menuId=040202>

3. ESG 평가 기준 관련

한국 ESG 기준원 (K-ESG)

<https://www.cgs.or.kr/main/main.jsp>

MSCI – 글로벌 ESG 평가 리서치

<https://www.msci.com/web/msci/research/esg-research>

4. 에너지 및 시장 현황

Knowable Magazine – 나이지리아 마을 태양광 사례(자료6)

<https://knowablemagazine.org/content/article/technology/2025/nigeria-bets-on-solar-minigrids-for-climate-livelihood>

재생에너지 클라우드 플랫폼 (태양광 발전소 개수)

<https://recloud.energy.or.kr/main/main.do>

한국에너지공단 – 신재생에너지 보급 실적 (PDF)(p.01)

<https://www.energy.or.kr/introduce/public/file/%EC%8B%A0%EC%9E%AC%EC%83%9D%EC%97%90%EB%84%88%EC%A7%80%EB%8C%80%EA%B5%AD%EB%AF%BC%EC%82%AC%EC%97%85%EB%B3%B4%EA%B8%89%EC%8B%A4%EC%A0%81.pdf>

Solar power world – 태양광 시장 점유율

<https://www.solarpowerworldonline.com/2022/02/lg-to-exit-solar-panel-manufacturing-and-close-alabama-plant>

해줌 – 태양광 시장 점유율 및 컨설팅

<https://haezoom.com/econsulting/>

에너지경제연구원 – 계시별 요금제 개선방안 연구: 정연제, 박광수: 시간별 전력량 비율 (p.40)

https://www.keei.re.kr/pdfOpen.es?bid=0001&list_no=81918&seq=1

한국에너지공단 – 에너지 바우처 지원 현황

<https://www.energy.or.kr/front/conts/102017010000000.do>

하나금융경영연구소 – 국내 에너지저장장치(ESS) 현황 및 전망: 오유진: 국내 ESS시장 전망 (p.07)

<https://www.scribd.com/document/801447099>

Q&A

